

● LIVE

หลักสูตรอบรมออนไลน์

Rust Programming



for Beginners



เหมาะสำหรับผู้เริ่มต้น
เริ่มจาก 0 อย่างเข้าใจ



มีวิดีโอบันทึกการอบรม
ย้อนหลังให้ทุกวัน

4 วัน
12

ชั่วโมงเต็ม



สถาบันไอทีจีเนียส



Samit Koyom
สถาบันไอทีจีเนียส



วิทยากร



อ.สามิตร โกยม

ปริญญาโทคณะเทคโนโลยีสารสนเทศ
มหาวิทยาลัยเทคโนโลยีพระจอมเกล้าพระนครเหนือ



สถาบันไอทีจีเนียส

► Frontend

Angular, React, Vue, Next, Nuxt, Bootstrap, Tailwind CSS

► Backend

PHP, Python, Java, Kotlin, Go, Rust, NodeJS, NestJS, .NET

► Database

MySQL, PostgreSQL, MS SQL Server, MongoDB, Supabase

► Mobile

Java, Kotlin, Objective C, Swift, KMP, Cordova, Flutter ,
React Native, Expo

► DevOps

Git, Github, Gitlab, Docker, Kubernetes, Jenkins, CI/CD

www.itgenius.co.th

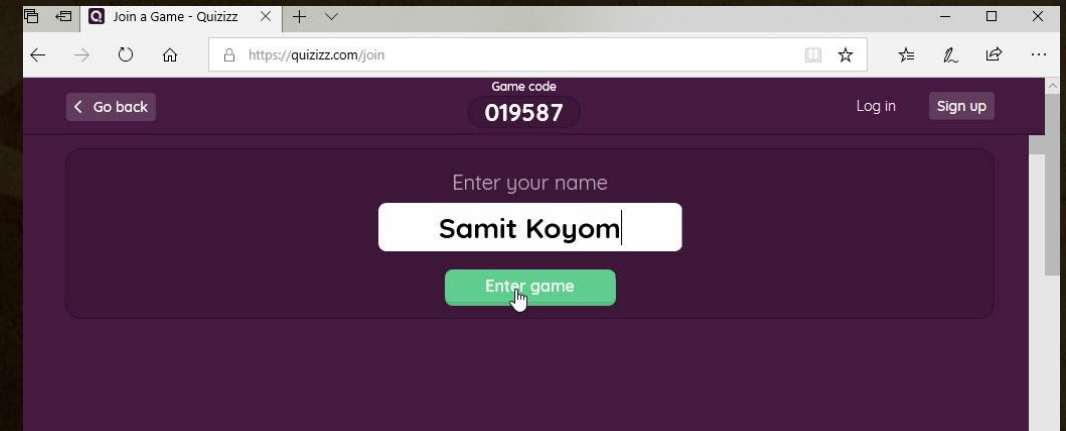
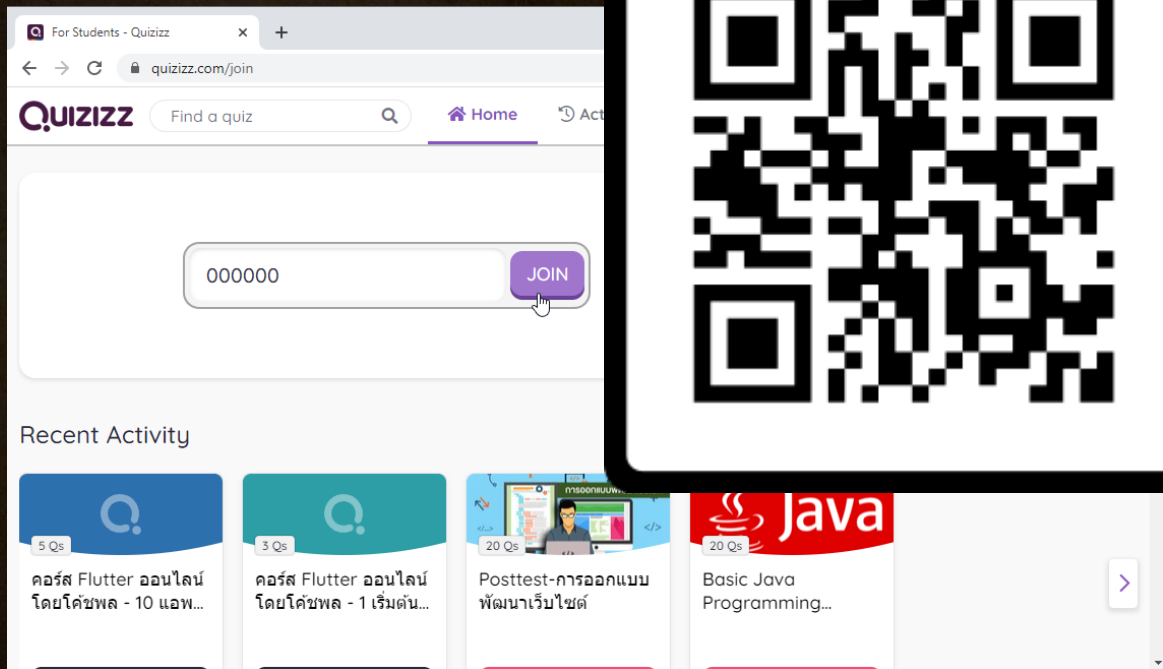
แบบทดสอบก่อนอบรม



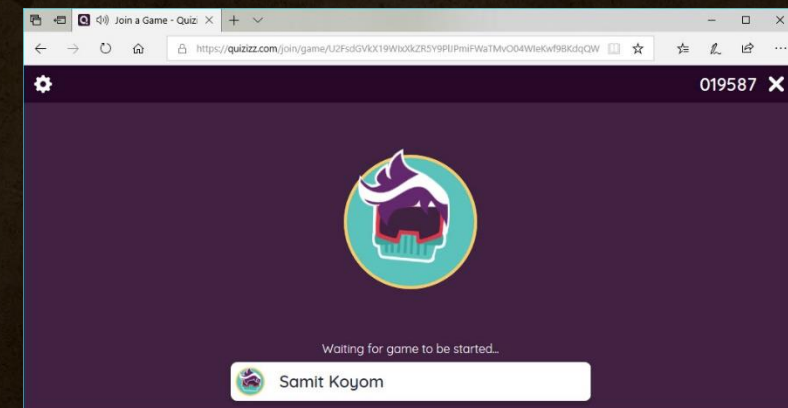
Pretest ทำแบบทดสอบก่อนเรียน

STEP 1: เข้าทำแบบทดสอบที่ลิงก์ ป้อนรหัสเข้าห้องสอบ **STEP 2:** ป้อนชื่อ

quizizz.com/join



STEP 3: รอผู้สอน Start ข้อสอบ



สถาบันไอทีจีเนียส

ninus.co.th



ดาวน์โหลดเอกสารประกอบการอบรม

bit.ly/beginner-rust



สถาบันไอทีจีเนียส

www.itgenius.co.th

Course Outline

Rust Programming for Beginners (4 Days)



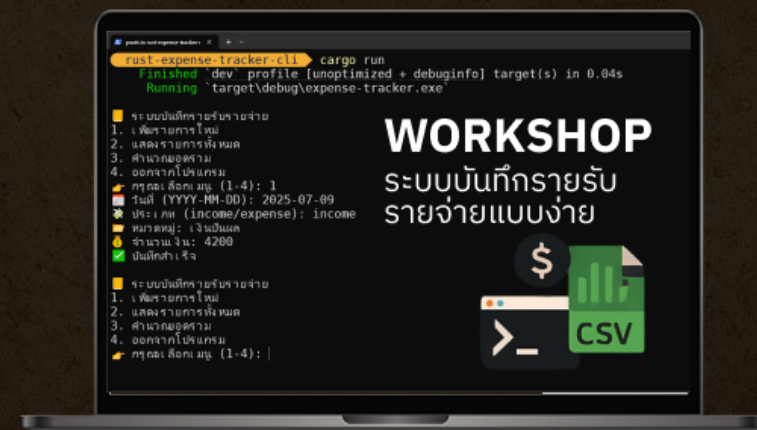
Section 1: พื้นฐานและการตั้งค่า Rust

Section 2: Control Flow & Ownership

Section 3: Collection & Error Handling

Section 4: โครงสร้างข้อมูลและ Mini Project

Section 5: Workshop: ระบบบันทึกรายรับรายจ่าย



● LIVE

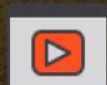
หลักสูตรอบรมออนไลน์

วันที่

1



Rust Programming for Beginners



มีวิดีโอบันทึกการอบรม
ย้อนหลังให้ทุกวัน



สอนสดผ่าน Zoom
รับจำนวนจำกัด

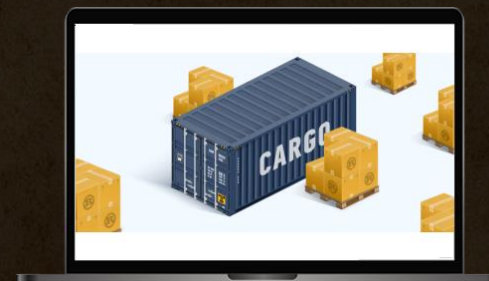


Samit Koyom
สถาบันไอทีจีเนียส

DAY 1

Section 1: พื้นฐานและการตั้งค่า Rust

Section 2: Control Flow & Ownership



สถาบันไอทีจีเนียส

www.itgenius.co.th

Section 1: พื้นฐานและการตั้งค่า Rust

- ประวัติและปรัชญาของ Rust
- รู้จัก Rust และจุดเด่น
- ติดตั้ง Rust, Cargo, VSCode
- การติดตั้งและตั้งค่า Rust toolchain
- การใช้งาน Cargo Package Manager
- การสร้างโปรเจกต์ Hello World ไวยากรณ์พื้นฐานและ data types
- การใช้งาน Rust Analyzer ใน VSCode
- การจัดการ Dependencies ด้วย Cargo.toml
- การใช้งาน Rustfmt และ Clippy สำหรับการจัดรูปแบบโค้ด
- ตัวแปร, ชนิดข้อมูล, ฟังก์ชัน
- แบบฝึกหัด: เขียนโปรแกรมง่าย ๆ





“ภาษาเก่า / เขาเก่าเกม / เคลมว่าแน่น
เขาของแตร / pointer ไป / แล้วไม่กลับ
malloc ลืม / ไม่ได้ free / นี่ไม่นับ
Segfault จับ / จอดสนิท / ชีวิตลอย”



“ภาษาจ๊าบ / เขียนโคตรง่าย / ใช้ไปทั่ว
บางทีมั่ว / เจอะอะไร / ให้อึ้งยัง (วะ)
ถึงลोजิก / ดีแค่ไหน / มันก็พัง
เพราะบ่อยครั้ง / ยัง NaN === NaN / แสบงเงใจ”



“พ่อเทพบุตร / สุด OOP / ดีสุดหล้า
แต่กินแรม / เยอะเสียกว่า / กินข้าวห่อ
บอก Write once / run ทุกที่ / ชอบรีรอ
ล่าช้าหนอ / เพราะ JVM / ไม่เต็มใจ”



“พอเจอ Rust / เหมือนฟ้าส่ง / ลงมาโปรด
ปลอดภัยโคตร / ใช้ memory / อย่างมีแผน
มี Ownership / กับ borrowing / ให้่งแทน
หมดปัญหา / Dangling แขนว / ไปได้เลย

ชูจุดเด่น / เน้น Zero cost / ปลด overhead
Generics ดี / มี Macro / ค่อยโวได้
C ยังบ่น / “unsafe” ฟรี / มีไม่ใช้
Java หงาย / Rust ของเด็ด / multi-thread เนียน”





ภาษา	จุดเด่น/สไตร์	ข้อจำกัด/จุดแขว	หมายเหตุเสริม
C	- เก้าเกม - ใช้งานไ	- เขาของแครง แต่ pointer ไปไม่กลับ - ลิม malloc / free - เจอ Segfault บ่อย	ต้องจัดการ memory เอง ไม่มีระบบความปลอดภัยเช่น ownership
Java	- ฟอเทพมด OOP - โครงสร้างชัดเจน	- กินแรมหนัก - Write once, run ก็จริงแต่รอ JVM แปลนนาน - ไม่เต็มใจเรื่อง performance	เหมาะกับระบบใหญ่ แต่มี JVM overhead
JavaScript	- เขียนโคตรง่าย - ใช้ได้ทุกที่ (web, server)	- == กับ === งงไปหมด - NaN === NaN = false - บักบ่อยแบบไม่มีที่มา	ดีสำหรับ frontend แต่ต้องระวัง coercion
Rust	- ฟาส่งมาช่วยโปรแกรมเมอร์ - ปลอดภัยสุด ๆ - มี ownership / borrowing - Zero-cost abstraction	- ต้องเรียนรู้เยอะหน่อยตอนเริ่ม - คอมไพล์ช้ากว่า แต่ได้ความปลอดภัยกลับมา	ประสิทธิภาพระดับ C แต่ปลอดภัยแบบยุคใหม่





ประวัติและปรัชญาของ Rust (History and Philosophy of Rust)



Rust Logo



Ferris
"Rustaceans"



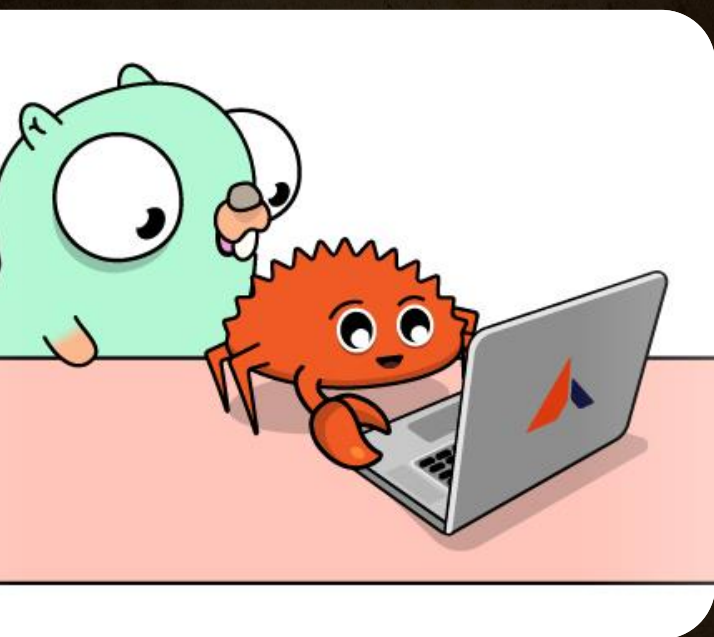
สถาบันไอทีจีเนียส

- **ผู้พัฒนา:**
 - ภาษา Rust เริ่มต้นโดย Graydon Hoare (Mozilla's Software Engineer) ในปี 2006 ระหว่างที่ทำงานอยู่กับ Mozilla
 - Mozilla เห็นศักยภาพและเริ่ม สนับสนุนอย่างจริงจังในปี 2009
- **เปิดตัวเวอร์ชัน 1.0:**
 - เมื่อวันที่ 15 พฤษภาคม 2015 หลังจากพัฒนาและปรับปรุงมาหลายปี
- **องค์กรที่ดูแลปัจจุบัน:**
 - ตั้งแต่ปี 2021 เป็นต้นมา Rust ดูแลโดยองค์กรไม่แสวงกำไรที่ชื่อว่า Rust Foundation
- **เวอร์ชันล่าสุด 1.88.0 (ตามข้อมูล ณ ปี 2025):**
 - Rust มีระบบปล่อยเวอร์ชันใหม่ทุก 6 สัปดาห์ และมีความมั่นคงทาง API ที่ดีมาก

www.itgenius.co.th



ปรัชญาของ Rust (Philosophy)



1. ความปลอดภัย (Safety)

- Rust ป้องกัน ปัญหาหน่วยความจำ ที่
- segmentation fault
- dangling pointer
- use-after-free

ผ่านกลไกของ ownership, borrowing, ตรวจสอบได้ขณะ compile

เป้าหมาย: ให้เขียนโค้ดที่ "ปลอดภัยจากขยะ garbage collector"



สถาบันไอทีจีเนียส

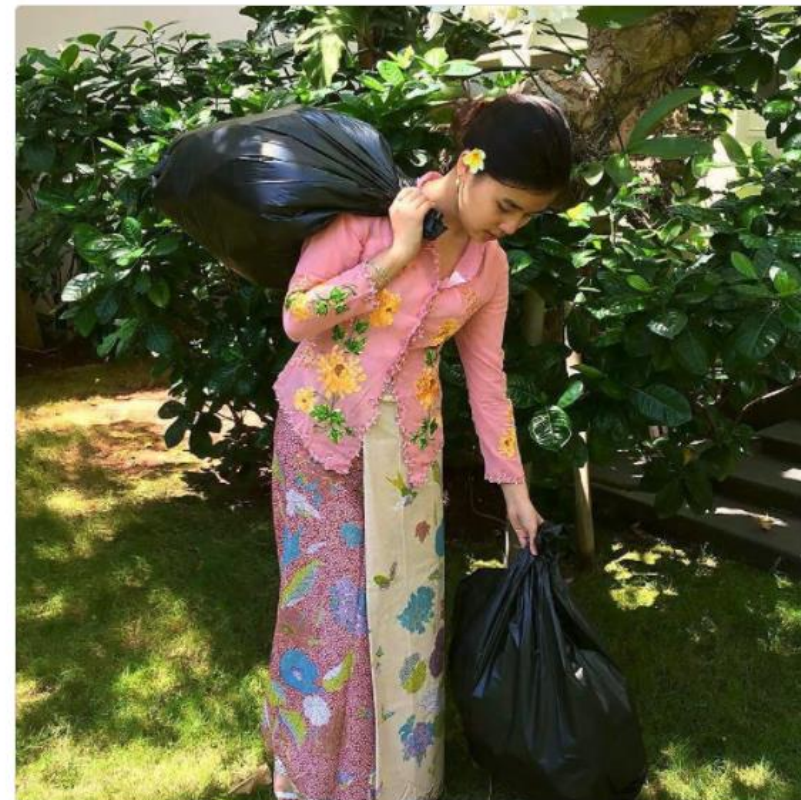


Jesslyn
@jtannady

Follow

I'm from the island of Java, Indonesia.

I am the Java Garbage Collector.



11:01 PM - 4 Apr 2018



ปรัชญาของ Rust (Philosophy)

Performance



2. ประสิทธิภาพสูง (High Performance)

- Rust สามารถ compile เป็น native code ที่ทำงานเร็ว เทียบเท่า C/C++
- ไม่มี garbage collector → ใช้งานในระบบฝังตัว, game engine หรือ WebAssembly ได้ดี
- ประหยัดพลังงานและ resource → เหมาะกับงานขนาดใหญ่ หรือรันบน server





ปรัชญาของ Rust (Philosophy)

Channels

Threads

Mutex



Rust

3. การเขียนโปรแกรมพร้อมกัน (Concurrency)

- การเขียนโปรแกรมแบบ multi-threading ที่ปลอดภัยเป็นจุดเด่น
- ป้องกัน data race ตั้งแต่ compile time
- สไลแกนที่นิยมใช้:
- “Fearless Concurrency” – เขียนโปรแกรมแบบ concurrent โดยไม่กลัว bug





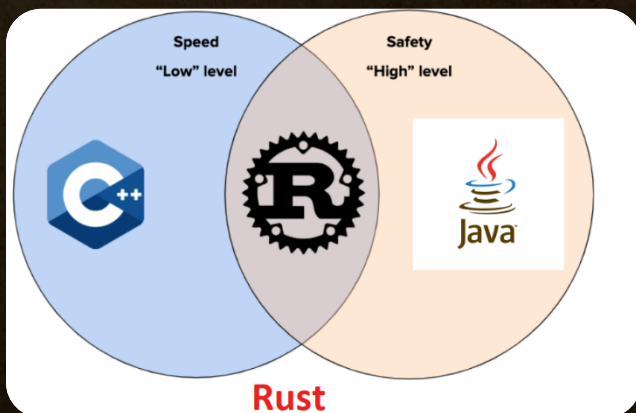
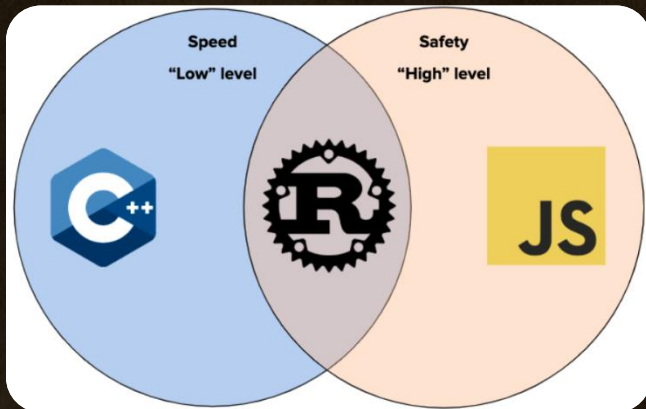
จุดแข็งของ Rust เทียบกับภาษาอื่น

ด้าน	Rust	C/C++	Go
Memory Safety	✓ ดีเยี่ยม (ไม่มี GC)	✗ เสี่ยงสูง	✓ มี GC
Compile-time Checking	✓ เข้มงวด	✗ น้อย	✓ ปานกลาง
Performance	✓ เทียบเท่า C	✓ สูง	⚠ ต่ำกว่า
Async/Concurrency	✓ ปลอดภัยและมี await	⚠ ใช้อยาก	✓ ง่าย
Learning Curve	⚠ สูง	⚠ สูง	✓ ต่ำ





คำขวัญของภาษา Rust



"Fast, reliable, productive: pick three."

แปลว่า: Rust พยายามให้คุณได้ทั้ง เร็ว (Fast), น่าเชื่อถือ (Reliable), และ เขียนง่าย/พัฒนาเร็ว (Productive) พร้อมกัน ซึ่งปกติภาษาอื่นมักต้องเลือกแค่ 2 อย่าง





กลยุทธ์เพิ่มประสิทธิภาพใน Rust



- **Zero-Cost Abstractions:** ใช้นามธรรม (เช่น iterator, generic) โดยไม่เสีย performance
- **Avoid Unnecessary Copies:** ย้าย (move) แทน copy หากไม่จำเป็น
- **Inline Functions:** ช่วยลด overhead ในการเรียกฟังก์ชัน
- **Use Efficient Data Structures:** เช่นใช้ VecDeque แทน Vec ถ้าเพิ่ม/ลบจากหัวท้ายบ่อย
- **Memory Management:** ใช้ ownership & borrowing ให้เหมาะสม
- **Leverage Concurrency:** ใช้ thread, async ให้ปลอดภัยและเกิดประสิทธิภาพ
- **Profile Your Code:** ใช้ cargo bench, perf, หรือ flamegraph ตรวจสอบวัดประสิทธิภาพจริง
- **Minimize Bounds Checking:** ใช้ get_unchecked() เมื่อมั่นใจว่าปลอดภัย (unsafe)





รู้จัก Rust และจุดเด่น (What is Rust and Its Strengths)

Rust เป็นภาษาโปรแกรม ระบบ (system programming language) ที่ ออกแบบมาเพื่อ:

- ✓ ความเร็ว (Performance)
- ✓ ความปลอดภัย (Memory Safety)
- ✓ ความน่าเชื่อถือ (Reliability)
- ✓ ความสามารถในการเขียนโค้ดที่ทำงานร่วมกัน (Concurrency)

Rust ถูกออกแบบมาให้ “เร็วพอ ๆ กับ C++ แต่ปลอดภัยกว่าโดยไม่ต้องใช้ Garbage Collector”





Rust ใช้ทำอะไรได้บ้าง?

ประเภทแอปพลิเคชัน

ตัวอย่าง

Embedded Systems

IoT, Arduino, Raspberry Pi

Command-line tools (CLI)

ripgrep, bat, exa

WebAssembly

แอปบนเบราว์เซอร์ที่ต้องการความเร็วสูง

Backend Web Services

ด้วย Framework เช่น Rocket, Actix Web

Operating Systems / Kernel

Redox OS, Firecracker (VM by AWS)

Game Development

ใช้ใน engine ที่ต้องการประสิทธิภาพสูง





ตัวอย่าง Rust framework for Web Development

ลำดับ	Framework	จุดเด่นสำคัญ
1	Actix Web	- ประสิทธิภาพสูงสุดในหลาย benchmark - ใช้ actor model รองรับ WebSocket, HTTP/2 - มี middleware ครบ เช่น logging, auth - routing แบบ type-safe
2	Axum	- พัฒนาโดยทีม Tokio - ใช้ Hyper และ Tower ได้ - routing แบบ type-safe ใช้ง่าย - รองรับ WebSocket และ SSE
3	Rocket	- API ระดับสูง เหมาะกับ rapid dev - ตรวจสอบพารามิเตอร์และ type ตอน compile - รองรับ templating, cookie, auth - ออกแบบมาเน้นความปลอดภัย
4	Warp	- แนวคิด composable filter - ใช้ async เต็มรูปแบบ - รองรับ WebSocket, error handling, auth - เบา เหมาะกับ microservice
5	Tide	- โครงสร้างเรียบง่าย เข้าใจง่าย - รองรับ middleware, async-ready - เหมาะกับผู้เริ่มต้น - ชุมชนกำลังเติบโต



Framework ทางเลือกเสริมที่น่าสนใจ

Framework ทางเลือก

ลำดับ	Framework	จุดเด่นสำคัญ
★	Salvo	- โครงสร้าง modular - รองรับ REST, WebSocket, gRPC - มี auth/session ในตัว - ใช้งานง่ายพร้อมเอกสารครบถ้วน





จุดเด่นของภาษา Rust

1. ไม่มี Garbage Collector แต่ยังปลอดภัย

Rust ใช้ระบบ Ownership และ Borrowing ที่ช่วยให้:

- ป้องกัน memory leak และ dangling pointers
- ป้องกัน data race และ thread safety ได้ตั้งแต่ compile time
- ทำงานได้ดีในระบบ embedded และ WebAssembly ที่ไม่มี resource มาก

rust

 Copy  Edit

```
fn main() {  
    let s = String::from("hello");  
    takes_ownership(s); // s ถูก move ไปแล้ว จะใช้อีกไม่ได้!  
    // println!("{}", s);  Error: value borrowed after move  
}  
  
fn takes_ownership(text: String) {  
    println!("{}", text);  
}
```





จุดเด่นของภาษา Rust

2. 🧠 การจัดการหน่วยความจำแบบ "Zero-Cost Abstraction"

- Rust ให้ abstraction สูง เช่น iterator, closure, trait แต่ **ไม่ลด performance**
- compiler จะ optimize ให้โดยไม่ต้องพึ่ง runtime

rust

📄 Copy ✎ Edit

```
let numbers = vec![1, 2, 3];  
let doubled: Vec<i32> = numbers.iter().map(|x| x * 2).collect();
```





จุดเด่นของภาษา Rust

3. ⚡ Compile-time Safety – ป้องกันบั๊กตั้งแต่ยังไม่รัน

- บั๊กที่เจอได้ใน runtime ของภาษาอื่น เช่น null pointer, race condition, dangling pointer
→ Rust ตรวจตั้งแต่ compile time

rust

📄 Copy ✎ Edit

```
fn main() {  
    let x: Option<i32> = None;  
    println!("{}", x.unwrap()); // ❌ Compile-time panic  
}
```





จุดเด่นของภาษา Rust

4. 🗝️ Concurrency ปลอดภัย (Fearless Concurrency)

- Rust สนับสนุน multi-threading แบบไม่กลัว data race
- borrow checker จะบังคับให้ใช้ thread-safe types เช่น `Arc`, `Mutex`

rust

📄 Copy 🖋️ Edit

```
use std::thread;

fn main() {
    let handle = thread::spawn(|| {
        println!("ทำงานใน thread ใหม่!");
    });

    handle.join().unwrap();
}
```





จุดเด่นของภาษา Rust

5. 📦 มีระบบ Package และ Build Tool ที่ยอดเยี่ยม: `cargo`

- สร้างโปรเจกต์, build, test, run ได้ง่ายในคำสั่งเดียว
- ระบบ dependency management ใช้ `Cargo.toml` แบบ declarative

bash

📄 Copy ✎ Edit

```
cargo new my_project  
cd my_project  
cargo run
```





จุดเด่นของภาษา Rust

6. สนับสนุน Testing และ Documentation

- เขียน test พร้อมโค้ดได้ในไฟล์เดียว
- ใช้คำสั่ง `cargo test` ในการทดสอบ
- สร้างเอกสารจาก comment โดยใช้ `cargo doc`

rust

 Copy  Edit

```
/// คูณสองค่าที่รับเข้ามา
fn double(x: i32) -> i32 {
    x * 2
}

#[test]
fn test_double() {
    assert_eq!(double(2), 4);
}
```





Rust เหมาะกับใคร

กลุ่มผู้ใช้	เหตุผลที่ควรใช้ Rust
นักพัฒนาระบบ	ต้องการความเร็วแบบ native พร้อม safety
ฝั่ง DevOps / CLI Tool	ต้องการ binary ขนาดเล็กและเร็ว
Web Developer	เขียน WASM เพื่อใช้ร่วมกับ JS
Embedded Developer	เขียน code ให้กับอุปกรณ์ขนาดเล็ก





Rust Community และ Tooling

- Community เป็นมิตร (friendly)
- มี tooling ครบ:
 - rustfmt (จัดรูปแบบโค้ด)
 - clippy (แนะนำสโตน์และ code smell)
 - cargo (build/test/package)
 - rust-analyzer (autocomplete, go to def ใน VSCode)





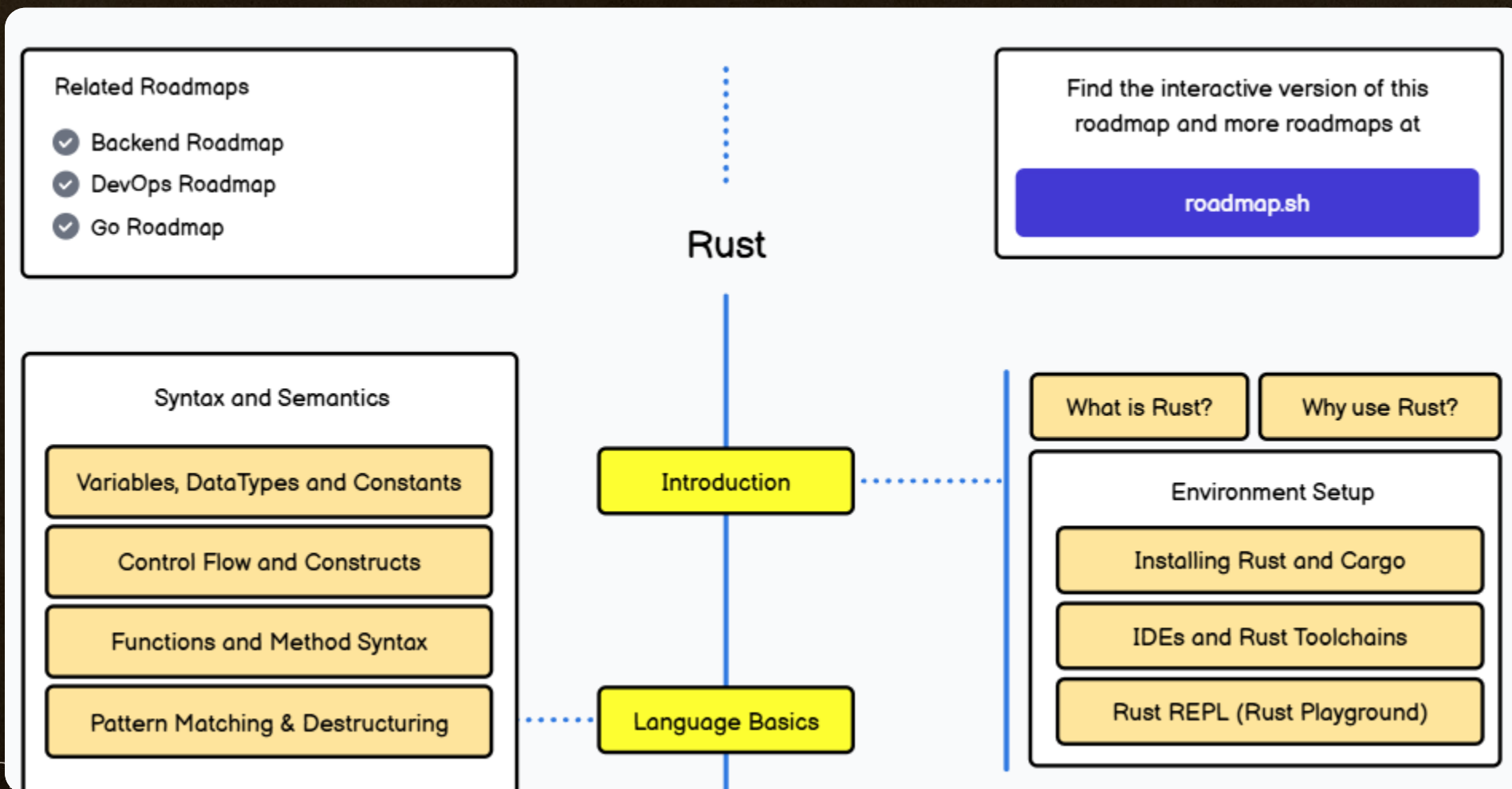
Top 10 บริษัทที่ใช้ Rust จริงใน Production

ลำดับ	บริษัท	การใช้งาน Rust
1	Amazon (AWS)	ใช้ Rust ในทีม Firecracker ซึ่งเป็น VMM (Virtual Machine Monitor) เบา ๆ ที่อยู่เบื้องหลัง AWS Lambda และ Fargate
2	Microsoft	ใช้ Rust ในโครงการด้าน Windows, Azure และบางส่วนใน security-critical components โดยมีการทดลองใช้แทน C/C++
3	Google	ใช้ Rust ในโครงการ Android Open Source Project (AOSP) เพื่อพัฒนา component ที่ปลอดภัยกว่า C++
4	Dropbox	ใช้ Rust แทน Python/C++ ใน core sync engine เพื่อความเร็วและ memory safety
5	Cloudflare	ใช้ Rust สำหรับ edge services (e.g. proxy, TLS handling) เพราะสามารถทำงานได้แบบ real-time และปลอดภัย
6	Mozilla	ต้นกำเนิดของ Rust ใช้ Rust ใน Firefox (เช่น Quantum Rendering Engine: Servo, Stylo)
7	Discord	ย้ายระบบบางส่วนจาก Go/Python ไปเป็น Rust โดยเฉพาะ Voice subsystem ที่ต้องการ latency ต่ำ
8	Facebook (Meta)	ใช้ Rust ในทีม Diem Blockchain (เดิม) และมี internal tools อื่น ๆ ที่ใช้ Rust
9	Tesla	ใช้ Rust ในระบบ backend และบางส่วนใน embedded system ที่ต้องการ real-time safety
10	Coursera / npm / npmjs	ใช้ Rust ในเครื่องมือ CLI, data pipeline, และ server-side components เพื่อ performance ที่ดีกว่า





Rust Developer Roadmap

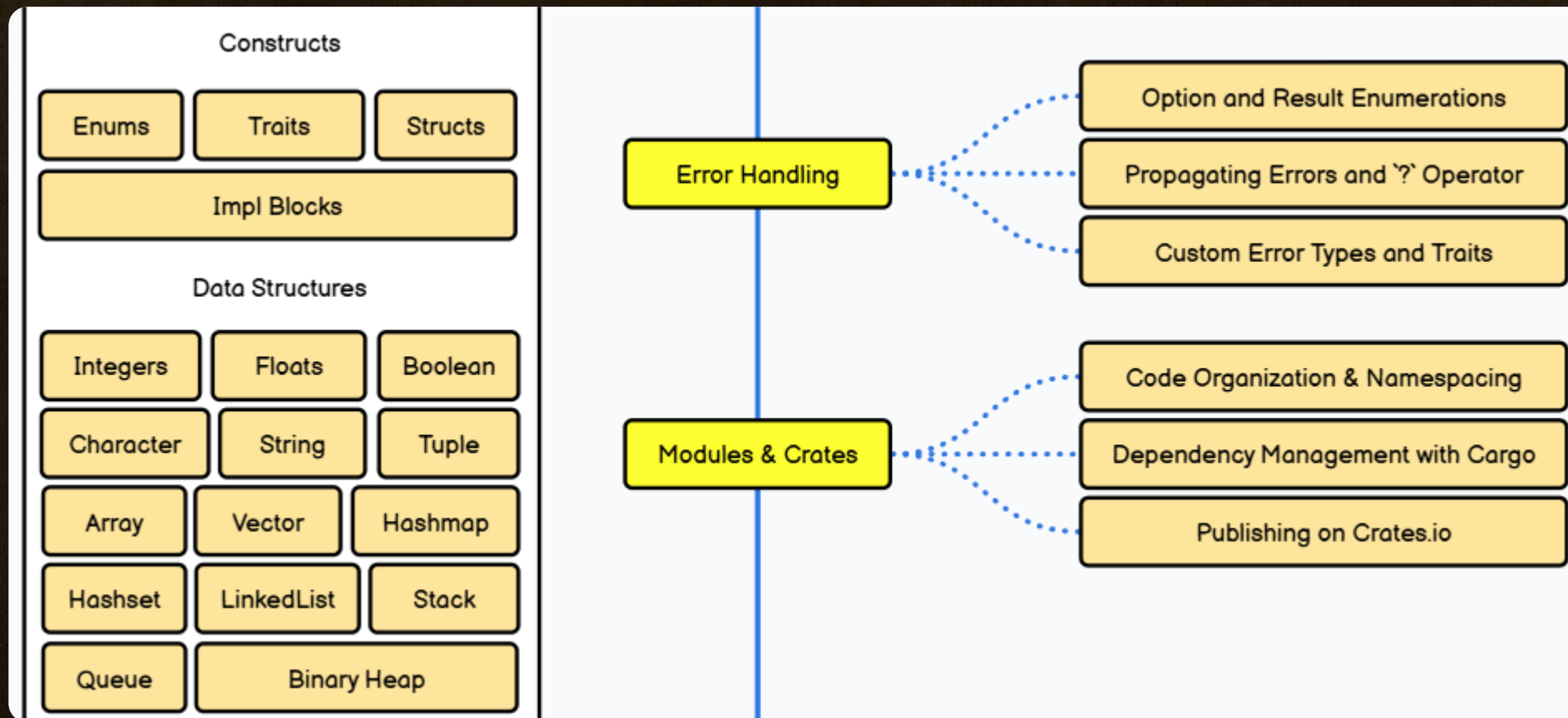


at

s.co.th

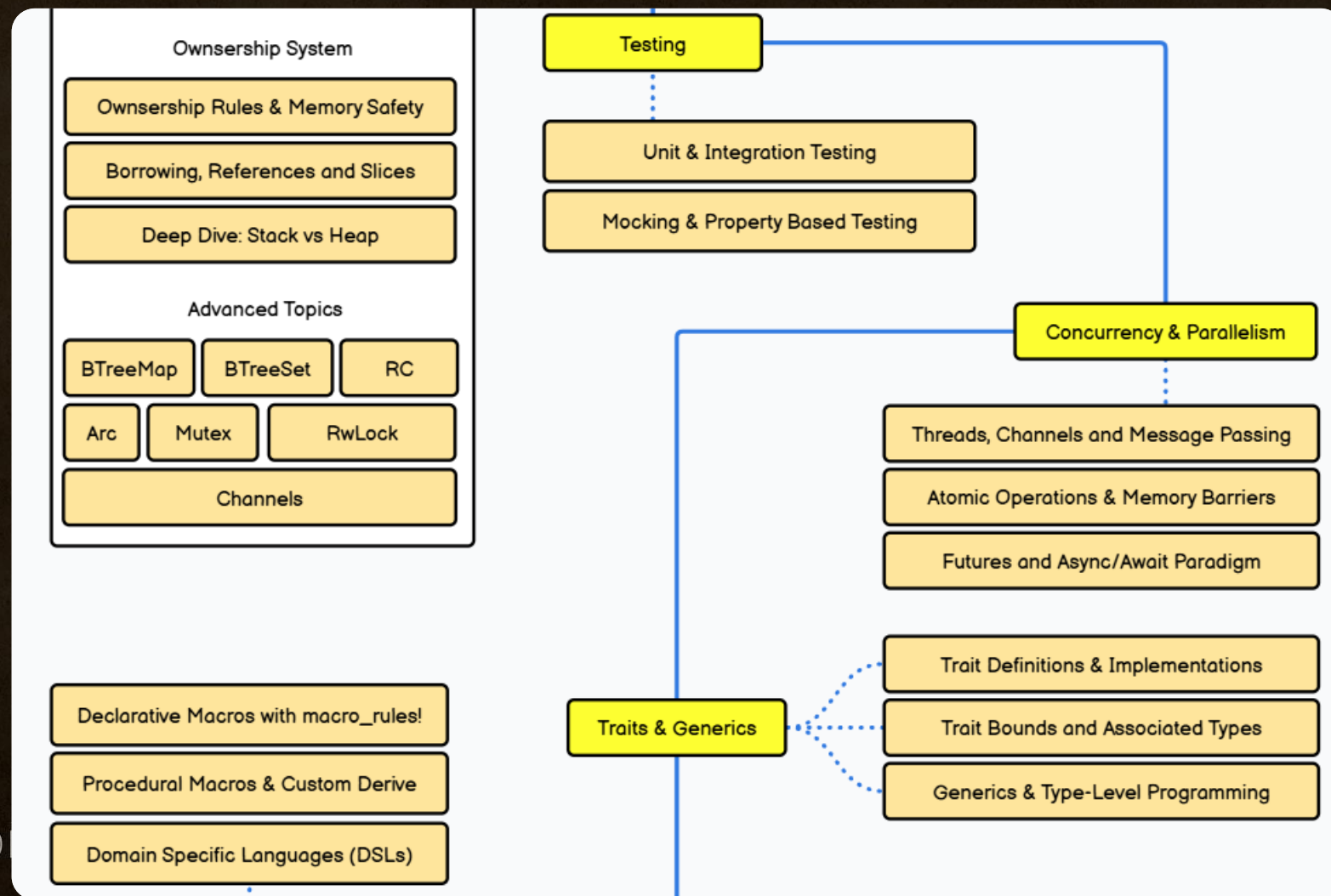


Rust Developer Roadmap





Rust Developer Roadmap

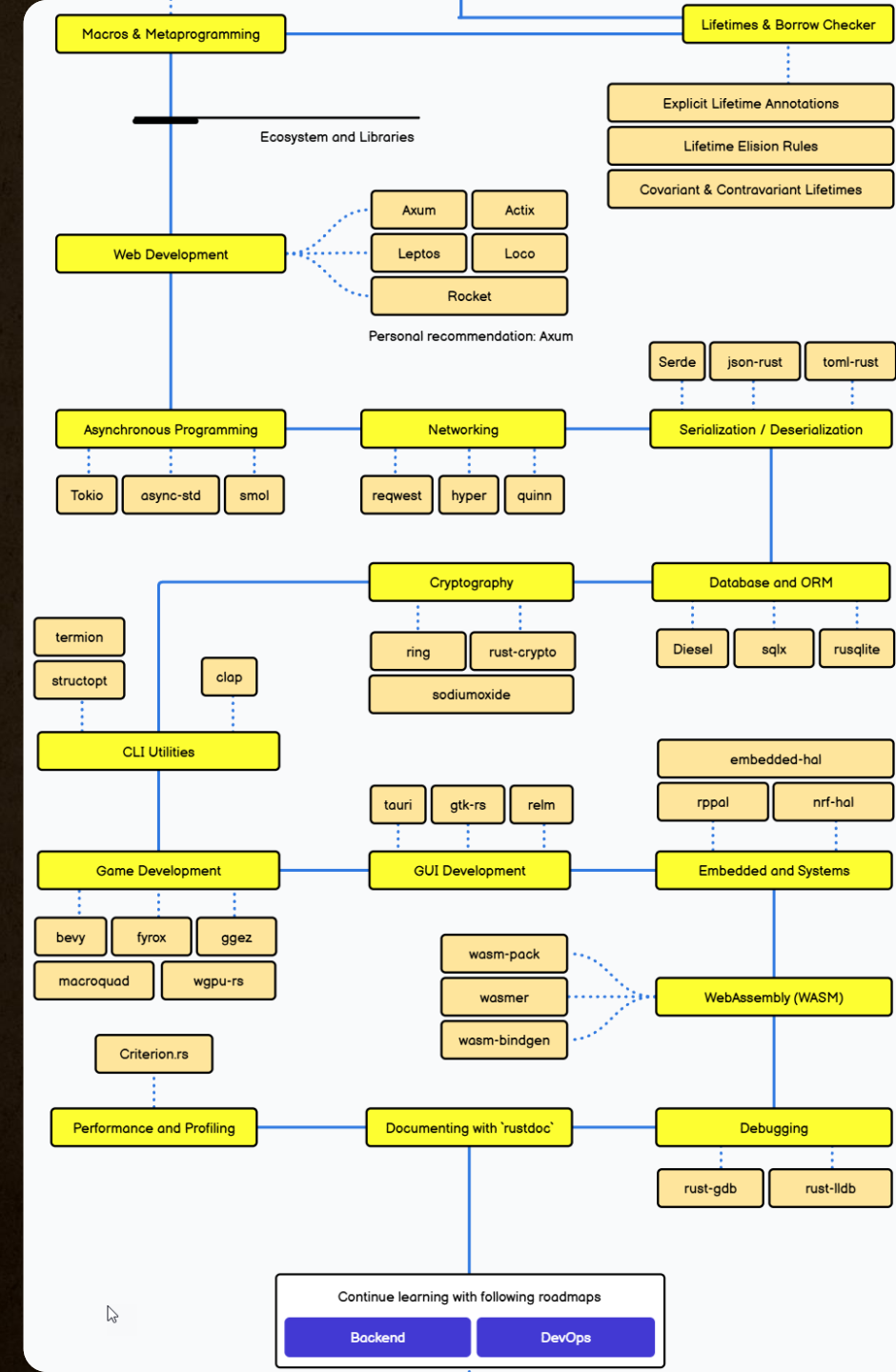




Rust Developer Roadmap



สถาบันไอทีจีเนียส



● LIVE



การเตรียมเครื่องมือ

Rust Programming



for Beginners



มีวิดีโอบันทึกการอบรม
ย้อนหลังให้ทุกวัน

4 วัน
12

ชั่วโมงเต็ม



สถาบันไอทีจีเนียส



Samit Koyom

สถาบันไอทีจีเนียส

System Requirement



Operating Systems

- Windows 10 Version 1607 and Windows 11
- Windows Server 2008 R2 SP1 (Full Server or Server Core)
- Windows Server 2012 SP1 (Full Server or Server Core)
- Windows Server 2012 R2 (Full Server or Server Core)
- Windows Server 2016 (Full Server, Server Core, or Nano Server)
- Mac OS 13.x, 14.x, 15.x, 26.x
- Red Hat Enterprise Linux 7
- Ubuntu 14.04, 16.04, 18.x, 20.x, 22.x, 24.x



Hardware Environment

- Processor: x86 or x64
- RAM : 4 MB (minimum), 8 GB (recommended)
- Hard disc: up to 3 GB of free space may be required



โปรแกรม (Tool and Editor) ที่ใช้บน

1. Rustup (version manager + toolchain installer)
2. Visual Studio Code
3. Git





Rust, Cargo และ VSCode คืออะไร ?

เครื่องมือ	รายละเอียด
Rust	ภาษาโปรแกรมที่ เราจะใช้พัฒนาโปรแกรม
Cargo	ตัวจัดการแพ็คเกจ (Package Manager) และ Build System ของ Rust (มาพร้อมกับ Rust)
VSCode	Visual Studio Code คือโปรแกรมเขียนโค้ดที่ใช้ร่วมกับ Rust ได้ดี





Rustup

(version manager + toolchain installer)



บนระบบ Windows



สถาบันไอทีจีเนียส

www.itgenius.co.th

Rustup คืออะไร ?

rustup คือ ตัวจัดการเวอร์ชัน (version manager) และเครื่องมือ (toolchain installer) สำหรับภาษา Rust ที่พัฒนาโดยทีมงานเดียวกับ Rust โดยตรง ใช้สำหรับติดตั้ง จัดการ และอัปเดตเวอร์ชันของ Rust ได้อย่างง่ายดาย โดยเฉพาะในสภาพแวดล้อมที่ต้องใช้หลายเวอร์ชันของ Rust หรือเครื่องมือเสริมต่าง ๆ

 เมื่อติดตั้ง `rustup` แล้วจะได้อะไรบ้าง?

หลังจากติดตั้ง `rustup` แล้ว อาจารย์จะได้เครื่องมือสำคัญต่าง ๆ ดังนี้:

เครื่องมือ	หน้าที่
<code>rustc</code>	คอมไพเลอร์ของ Rust
<code>cargo</code>	ตัวจัดการ package/project (build, test, run)
<code>rustup</code>	ตัวจัดการเวอร์ชัน
<code>rustfmt</code>	ตัวจัด format code ให้เป็นมาตรฐาน
<code>clippy</code>	ตัวช่วยตรวจ style และปัญหาที่พบบ่อยในโค้ด Rust



ตรวจสอบเช็คว่ามีการติดตั้ง Rustup ไว้หรือยัง

พิมพ์คำสั่ง **rustc --version** ตรวจสอบ
หากขึ้นข้อความดังภาพ แสดงว่ายังไม่ได้ติดตั้ง rustup
(rust version manager + toolchain installer)

```
Command Prompt
C:\Users\Samit>rustc --version
rustc is not recognized as an internal or external command,
operable program or batch file.
C:\Users\Samit>
```

หากพบเวอร์ชันของ Rust ดังภาพ แสดงว่าติดตั้งแล้ว

```
Command Prompt
C:\Users\Samit>rustc --version
rustc 1.88.0 (6b00bc388 2025-06-23)
C:\Users\Samit>
```

ในหลักสูตรนี้ใช้ Rust ได้ตั้งแต่เวอร์ชัน 1.7.x+ หรือใหม่กว่า

หากต้องการลงเวอร์ชันใหม่ล่าสุด บน Windows สามารถใช้คำสั่งอัปเดตได้ดังตัวอย่างในหน้าถัดไป



หากมี Rustup ติดตั้งไว้แล้วอยากอัปเดตเป็นเวอร์ชันใหม่กว่า
ใช้คำสั่งอัปเดตดังนี้

```
C:\Users\Samit>rustup update
info: syncing channel updates for 'stable-x86_64-pc-windows-msvc'
info: checking for self-update

stable-x86_64-pc-windows-msvc unchanged - rustc 1.88.0 (6b00bc388 2025-06-23)

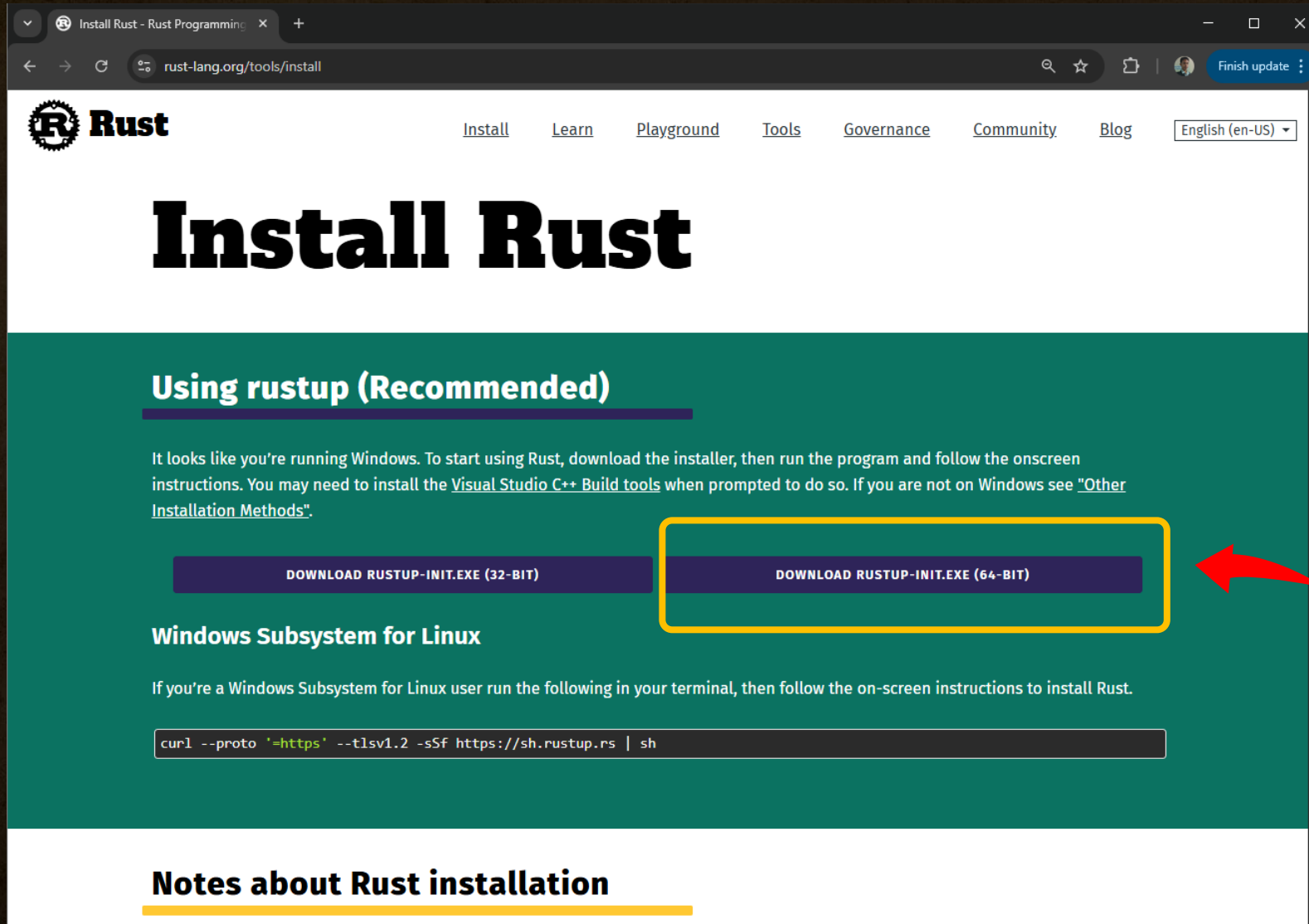
info: cleaning up downloads & tmp directories

C:\Users\Samit>rustc --version
rustc 1.88.0 (6b00bc388 2025-06-23)

C:\Users\Samit>_
```



สำรวจเว็บไซต์สำหรับดาวน์โหลด rustup



Install Rust - Rust Programming

rust-lang.org/tools/install

Rust

[Install](#) [Learn](#) [Playground](#) [Tools](#) [Governance](#) [Community](#) [Blog](#) [English \(en-US\)](#)

Install Rust

Using rustup (Recommended)

It looks like you're running Windows. To start using Rust, download the installer, then run the program and follow the onscreen instructions. You may need to install the [Visual Studio C++ Build tools](#) when prompted to do so. If you are not on Windows see "[Other Installation Methods](#)".

[DOWNLOAD RUSTUP-INIT.EXE \(32-BIT\)](#) [DOWNLOAD RUSTUP-INIT.EXE \(64-BIT\)](#)

Windows Subsystem for Linux

If you're a Windows Subsystem for Linux user run the following in your terminal, then follow the on-screen instructions to install Rust.

```
curl --proto 'https' --tlsv1.2 -sSf https://sh.rustup.rs | sh
```

Notes about Rust installation

ดาวน์โหลดไฟล์นี้
สำหรับ Windows



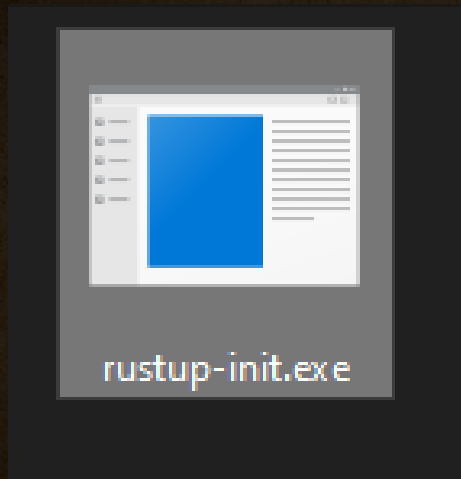
สถาบันไอทีจีเนียส

<https://www.rust-lang.org/tools/install>

www.itgenius.co.th

พิมพ์ 1 หรือกด Enter เพื่อเริ่มติดตั้งได้เลย

ไฟล์ Rustup ที่ดาวน์โหลดมา
ทำการคลิกเริ่มติดตั้ง



```
C:\Users\Samit\Downloads\rustup-init.exe

The Cargo home directory is located at:

C:\Users\Samit\.cargo

This can be modified with the CARGO_HOME environment variable.

The cargo, rustc, rustup and other commands will be added to
Cargo's bin directory, located at:

C:\Users\Samit\.cargo\bin

This path will then be added to your PATH environment variable by
modifying the PATH registry key at HKEY_CURRENT_USER\Environment.

You can uninstall at any time with rustup self uninstall and
these changes will be reverted.

Current installation options:

    default host triple: x86_64-pc-windows-msvc
    default toolchain: stable (default)
    profile: default
    modify PATH variable: yes

1) Proceed with standard installation (default - just press enter)
2) Customize installation
3) Cancel installation
>
```



สถาบันไอทีจีเนียส

หน้าจอเมื่อติดตั้ง rustup สำเร็จเรียบร้อยแล้ว

```
C:\Users\Samit\Downloads\rustup-init.exe

info: profile set to 'default'
info: default host triple is x86_64-pc-windows-msvc
info: syncing channel updates for 'stable-x86_64-pc-windows-msvc'
info: latest update on 2025-06-26, rust version 1.88.0 (6b00bc388 2025-06-23)
info: downloading component 'cargo'
info: downloading component 'clippy'
info: downloading component 'rust-docs'
info: downloading component 'rust-std'
 20.3 MiB /  20.3 MiB (100 %) 19.1 MiB/s in  1s
info: downloading component 'rustc'
 75.7 MiB /  75.7 MiB (100 %) 22.3 MiB/s in  4s
info: downloading component 'rustfmt'
info: installing component 'cargo'
info: installing component 'clippy'
info: installing component 'rust-docs'
 20.1 MiB /  20.1 MiB (100 %)  2.3 MiB/s in  7s
info: installing component 'rust-std'
 20.3 MiB /  20.3 MiB (100 %) 15.2 MiB/s in  1s
info: installing component 'rustc'
 75.7 MiB /  75.7 MiB (100 %) 16.5 MiB/s in  4s
info: installing component 'rustfmt'
info: default toolchain set to 'stable-x86_64-pc-windows-msvc'

  stable-x86_64-pc-windows-msvc installed - rustc 1.88.0 (6b00bc388 2025-06-23)

Rust is installed now. Great!

To get started you may need to restart your current shell.
This would reload its PATH environment variable to include
Cargo's bin directory (%USERPROFILE%\cargo\bin).

Press the Enter key to continue.
```



หลังติดตั้งเสร็จทดสอบด้วยคำสั่งนี้

rustc --version

Cargo --version

```
Command Prompt

C:\Users\Samit>rustc --version
rustc 1.88.0 (6b00bc388 2025-06-23)

C:\Users\Samit>cargo --version
cargo 1.88.0 (873a06493 2025-05-10)

C:\Users\Samit>
```



ส

s.co.th



Rustup

(version manager + toolchain installer)



รองรับ MacOS / Linux

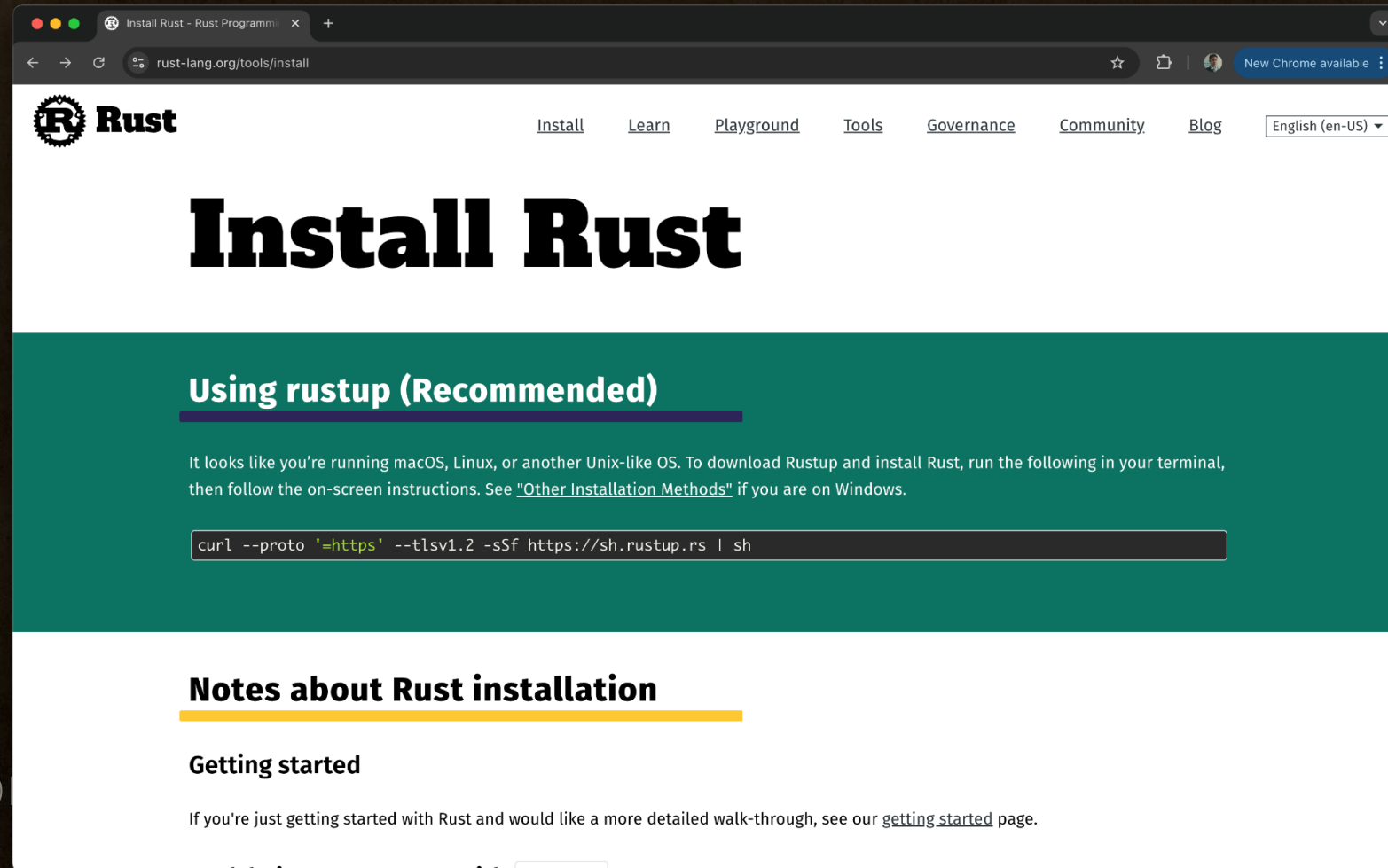


สถาบันไอทีจีเนียส

www.itgenius.co.th

สำหรับการติดตั้ง rustup บนระบบปฏิบัติการ MacOS หรือ Linux ให้คัดลอกคำสั่งด้านล่างนี้ไปรันใน Terminal ของ MacOS ได้เลย

```
curl --proto '=https' --tlsv1.2 -sSf https://sh.rustup.rs | sh
```

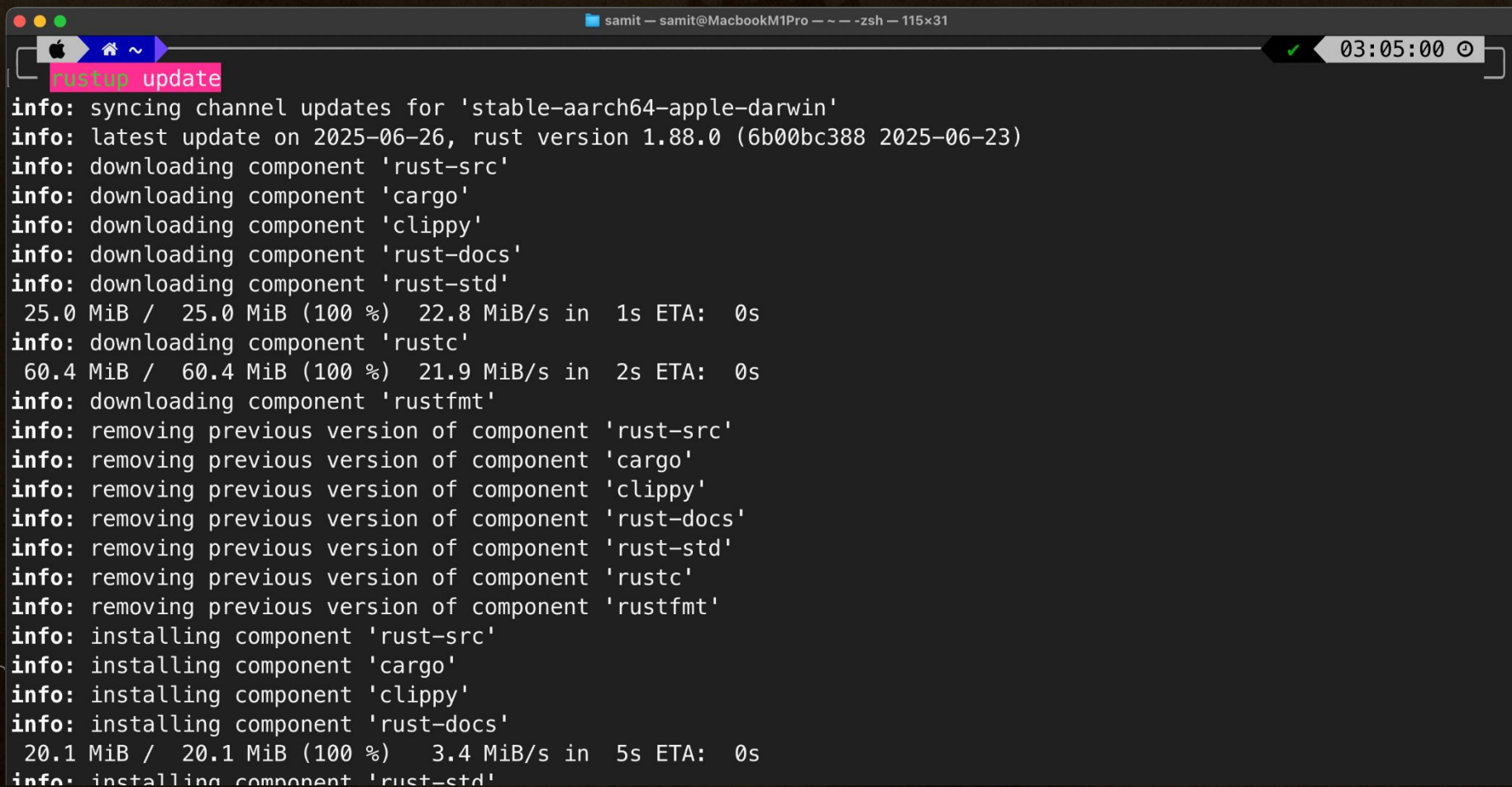


สถาบันไอ

itgenius.co.th

หากมีการติดตั้งไว้แล้วต้องการ อัปเดตบน MacOS หรือ Linux
ใช้คำสั่งนี้เช่นเดียวกัน

`rustup update`



```
samit — samit@MacbookM1Pro — ~ — zsh — 115x31
rustup update
info: syncing channel updates for 'stable-aarch64-apple-darwin'
info: latest update on 2025-06-26, rust version 1.88.0 (6b00bc388 2025-06-23)
info: downloading component 'rust-src'
info: downloading component 'cargo'
info: downloading component 'clippy'
info: downloading component 'rust-docs'
info: downloading component 'rust-std'
 25.0 MiB /  25.0 MiB (100 %) 22.8 MiB/s in  1s ETA:  0s
info: downloading component 'rustc'
 60.4 MiB /  60.4 MiB (100 %) 21.9 MiB/s in  2s ETA:  0s
info: downloading component 'rustfmt'
info: removing previous version of component 'rust-src'
info: removing previous version of component 'cargo'
info: removing previous version of component 'clippy'
info: removing previous version of component 'rust-docs'
info: removing previous version of component 'rust-std'
info: removing previous version of component 'rustc'
info: removing previous version of component 'rustfmt'
info: installing component 'rust-src'
info: installing component 'cargo'
info: installing component 'clippy'
info: installing component 'rust-docs'
 20.1 MiB /  20.1 MiB (100 %)  3.4 MiB/s in  5s ETA:  0s
info: installing component 'rust-std'
```



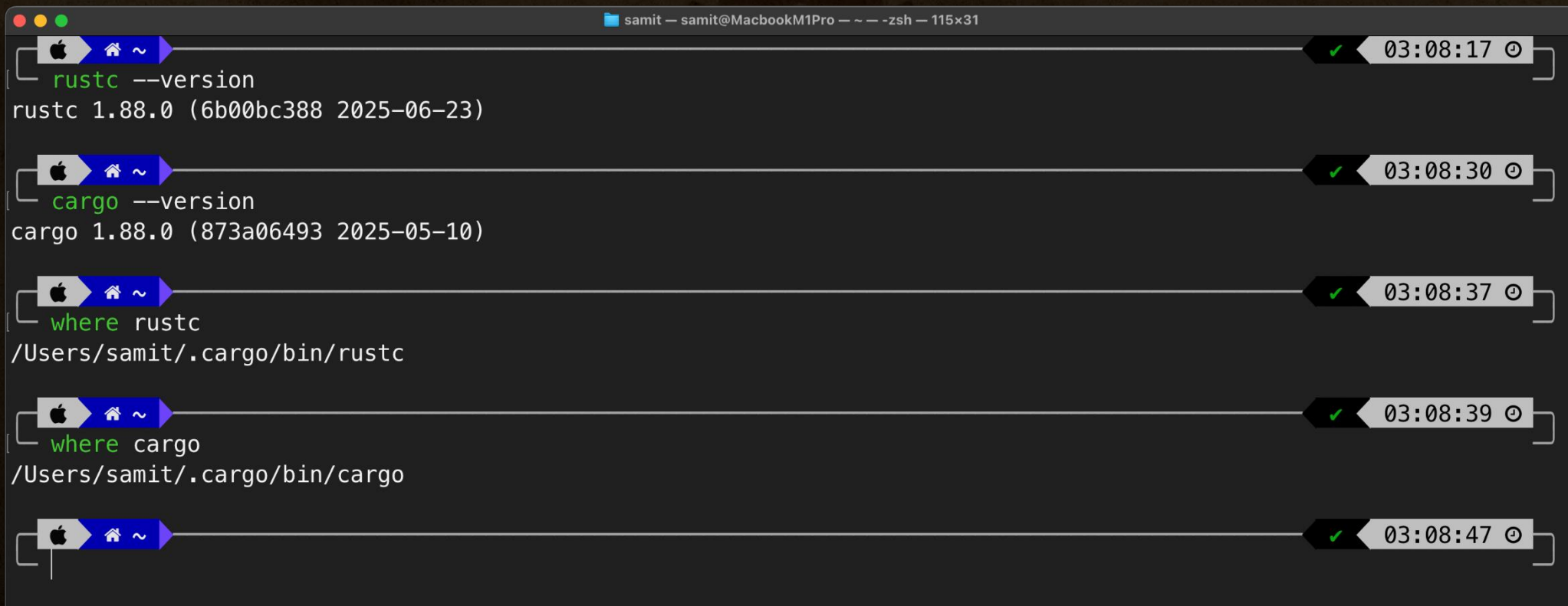
สกปรก

us.co.th

หลังติดตั้งเสร็จทดสอบด้วยคำสั่งนี้

`rustc --version`

`Cargo --version`



A terminal window titled 'samit — samit@MacbookM1Pro — — -zsh — 115x31' displays five commands and their outputs, each preceded by a progress bar and a timestamp. The commands and their outputs are:

- `rustc --version` outputs: `rustc 1.88.0 (6b00bc388 2025-06-23)` (03:08:17)
- `cargo --version` outputs: `cargo 1.88.0 (873a06493 2025-05-10)` (03:08:30)
- `where rustc` outputs: `/Users/samit/.cargo/bin/rustc` (03:08:37)
- `where cargo` outputs: `/Users/samit/.cargo/bin/cargo` (03:08:39)
- (No command shown, but timestamp 03:08:47 is present)



คำสั่งพื้นฐานของ rustup

คำสั่ง	ความหมาย
<code>rustup update</code>	อัปเดต toolchain ทั้งระบบให้เป็นเวอร์ชันล่าสุด
<code>rustup show</code>	แสดงข้อมูลเกี่ยวกับ toolchain ที่ใช้อยู่
<code>rustup default <เวอร์ชัน></code>	ตั้งค่าเวอร์ชัน default ของ Rust
<code>rustup override set <เวอร์ชัน></code>	ตั้งค่าเฉพาะในโปรเจกต์
<code>rustup target list</code>	แสดง platform ทั้งหมดที่รองรับ
<code>rustup component add <ชื่อ></code>	ติดตั้ง component เสริม (เช่น <code>rustfmt</code> , <code>clippy</code>)



ตัวอย่างการใช้งาน

✓ ตรวจสอบเวอร์ชันปัจจุบัน

bash

Copy Edit

```
rustc --version  
cargo --version  
rustup show
```

✓ อัปเดต Rust ทั้งระบบ

bash

Copy Edit

```
rustup update
```



ตัวอย่างการใช้งาน

✓ ติดตั้งเวอร์ชันอื่น (เช่น nightly หรือ beta)

bash

Copy Edit

```
rustup install nightly  
rustup install beta
```

ใช้สำหรับทดลองฟีเจอร์ใหม่ ๆ หรือ compile crate ที่ยังไม่ stable

✓ ตั้งค่าให้ใช้ nightly เฉพาะโปรเจกต์

bash

Copy Edit

```
cd my_project  
rustup override set nightly
```

หากไม่ต้องการใช้ nightly แล้ว:

bash

Copy Edit

```
rustup override unset
```



ติดตั้ง Component เสริม

1. `rustfmt` – เครื่องมือจัดรูปแบบโค้ด

bash

Copy Edit

```
rustup component add rustfmt  
cargo fmt
```

2. `clippy` – เครื่องมือวิเคราะห์คุณภาพโค้ด

bash

Copy Edit

```
rustup component add clippy  
cargo clippy
```

3. `rust-analyzer` – Language Server สำหรับ VSCode

- แนะนำให้ติดตั้งผ่าน Extension ของ VSCode โดยตรง
- VSCode จะใช้ toolchain ที่ `rustup` จัดการอัตโนมัติ



โครงสร้าง Toolchain หลายเวอร์ชัน

คุณสามารถมีได้หลายเวอร์ชันพร้อมกัน เช่น:

Project	Toolchain
/app-prod	stable
/app-research	nightly

โดยกำหนดได้ผ่านไฟล์ `rust-toolchain.toml` หรือ `rust-toolchain`

 ตัวอย่าง `rust-toolchain.toml`

toml

 Copy  Edit

```
[toolchain]
channel = "nightly"
components = ["rustfmt", "clippy"]
```

วางไฟล์นี้ไว้ใน root folder ของโปรเจกต์ → Rust จะใช้ nightly + component ที่กำหนดให้อัตโนมัติ



Component อะไร “ติดตั้งได้บ้าง” สำหรับ toolchain ปัจจุบัน

bash

Copy Edit

```
rustup component list --installed
```

หรือถ้าจะดูทั้งหมดที่ สามารถติดตั้งได้:

bash

Copy Edit

```
rustup component list
```

จะเห็นรายการ เช่น:

scss

Copy Edit

```
clippy-x86_64-unknown-linux-gnu (installed)
rust-src (installed)
rustfmt-x86_64-unknown-linux-gnu (installed)
llvm-tools-preview-x86_64-unknown-linux-gnu
...
```



คำสั่งที่น่าสนใจเพิ่มเติม

คำสั่ง

ความหมาย

```
rustup component add rustfmt
```

ติดตั้ง `rustfmt`

```
rustup component remove clippy
```

ถอนการติดตั้ง `clippy`

```
rustup show
```

แสดงสถานะ toolchain และ component ทั้งหมด

```
rustup target list --installed
```

แสดง target platform ที่ติดตั้งไว้ เช่น WASM





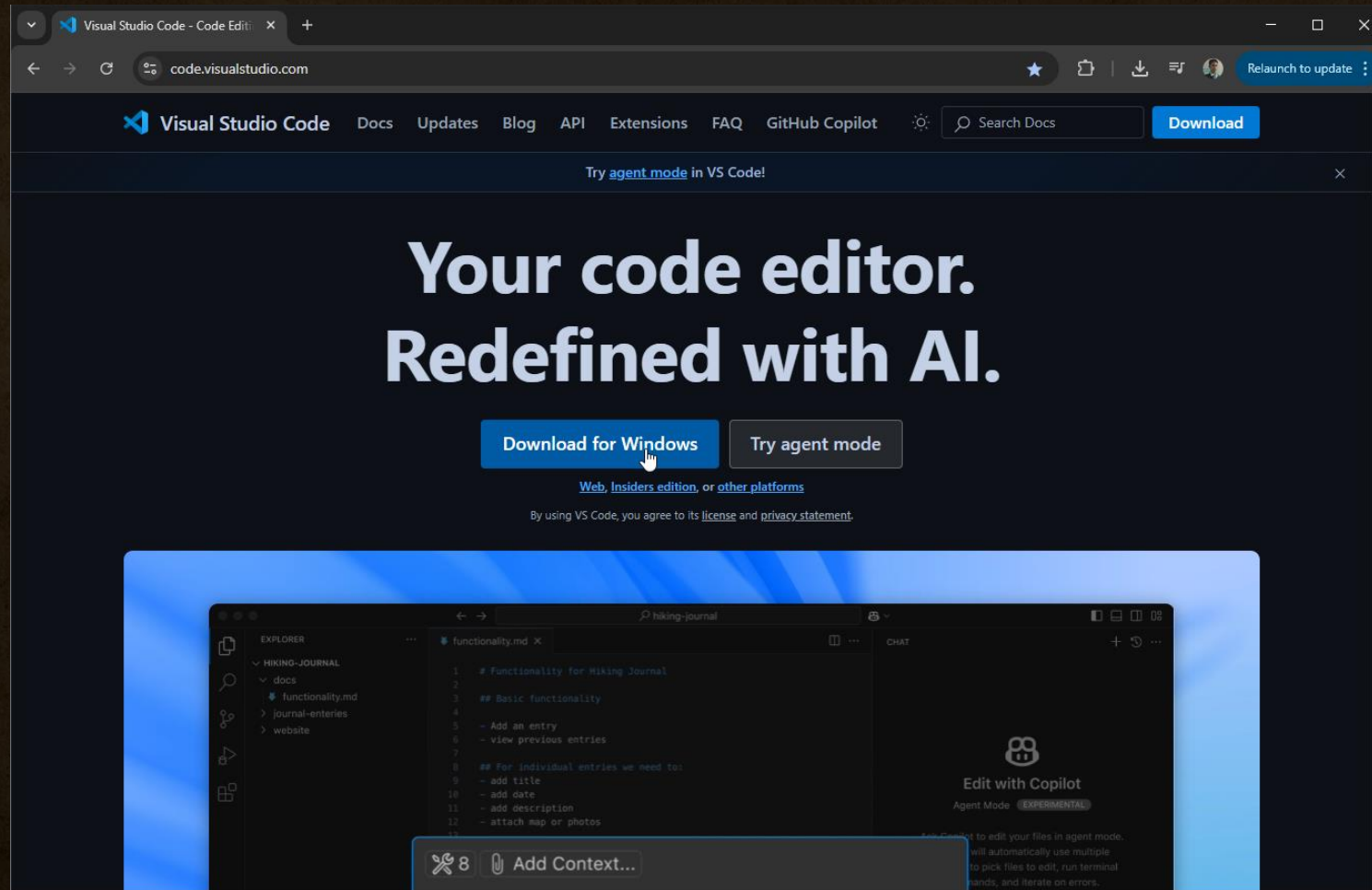
ติดตั้ง Visual Studio Code



สถาบันไอทีจีเนียส

www.itgenius.co.th

ติดตั้ง Visual Studio Code พร้อมส่วนเสริมที่จำเป็น



สถาบันไอทีจีเนียส

เข้าดาวน์โหลด Visual Studio Code ได้ที่ <https://code.visualstudio.com>

www.itgenius.co.th

การติดตั้งส่วนเสริม (Extension) ของ Visual Studio Code



สถาบันไอทีจีเนียส

www.itgenius.co.th

รายชื่อ Extensions ที่แนะนำสำหรับ VS Code

1. **rust-analyzer** by The Rust Programming Language
2. **Rust Syntax** by Dusty Pomerleau
3. **Material Icon Theme** by pkief.com
4. **One Dark Pro** by binaryify





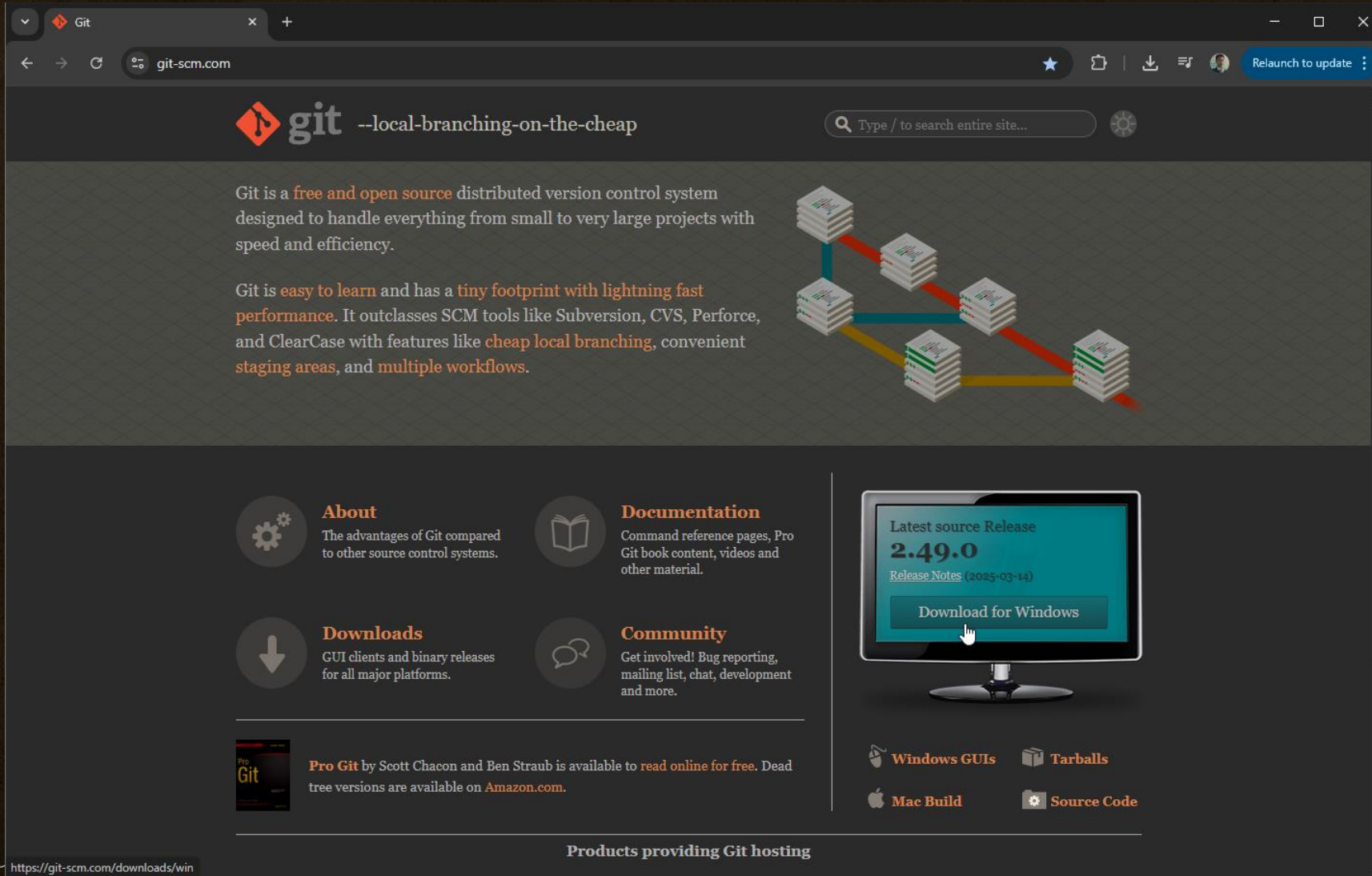
5. ติดตั้ง Git



สถาบันไอทีจีเนียส

www.itgenius.co.th

ดาวน์โหลดไฟล์ติดตั้ง Git ได้ที่ <https://git-scm.com/>



The screenshot shows the Git website homepage. At the top, there's a navigation bar with the Git logo and the tagline "--local-branching-on-the-cheap". A search bar is located on the right. The main content area features a description of Git as a free and open source distributed version control system, highlighting its speed, efficiency, ease of learning, and tiny footprint. To the right of the text is a diagram illustrating Git's branching model with stacks of code and colored lines representing branches. Below the main text, there are four sections: "About" (advantages of Git), "Documentation" (command reference, Pro Git book), "Downloads" (GUI clients and binary releases), and "Community" (bug reporting, mailing list). On the right side, there's a section for the "Latest source Release 2.49.0" with a "Download for Windows" button. At the bottom, there are links for "Windows GUIs", "Tarballs", "Mac Build", and "Source Code". A footer section lists "Products providing Git hosting".

Git is a **free and open source** distributed version control system designed to handle everything from small to very large projects with speed and efficiency.

Git is **easy to learn** and has a **tiny footprint with lightning fast performance**. It outclasses SCM tools like Subversion, CVS, Perforce, and ClearCase with features like **cheap local branching**, convenient staging areas, and **multiple workflows**.

About
The advantages of Git compared to other source control systems.

Documentation
Command reference pages, Pro Git book content, videos and other material.

Downloads
GUI clients and binary releases for all major platforms.

Community
Get involved! Bug reporting, mailing list, chat, development and more.

Latest source Release
2.49.0
[Release Notes \(2025-03-14\)](#)
[Download for Windows](#)

Windows GUIs **Tarballs**
Mac Build **Source Code**

Products providing Git hosting

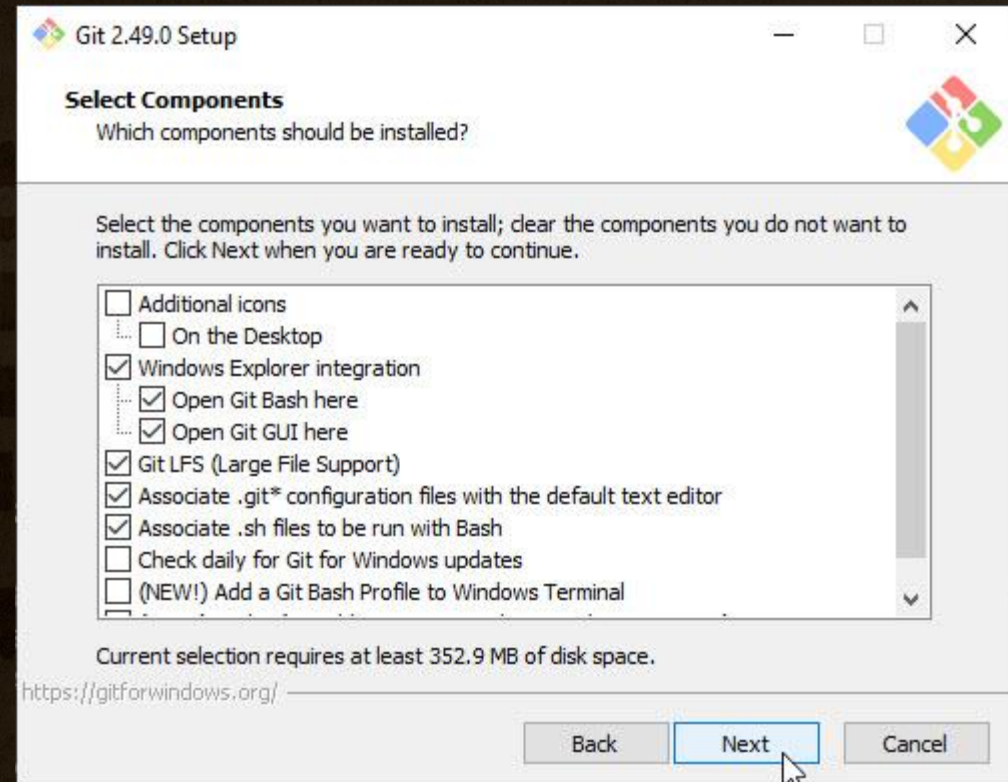
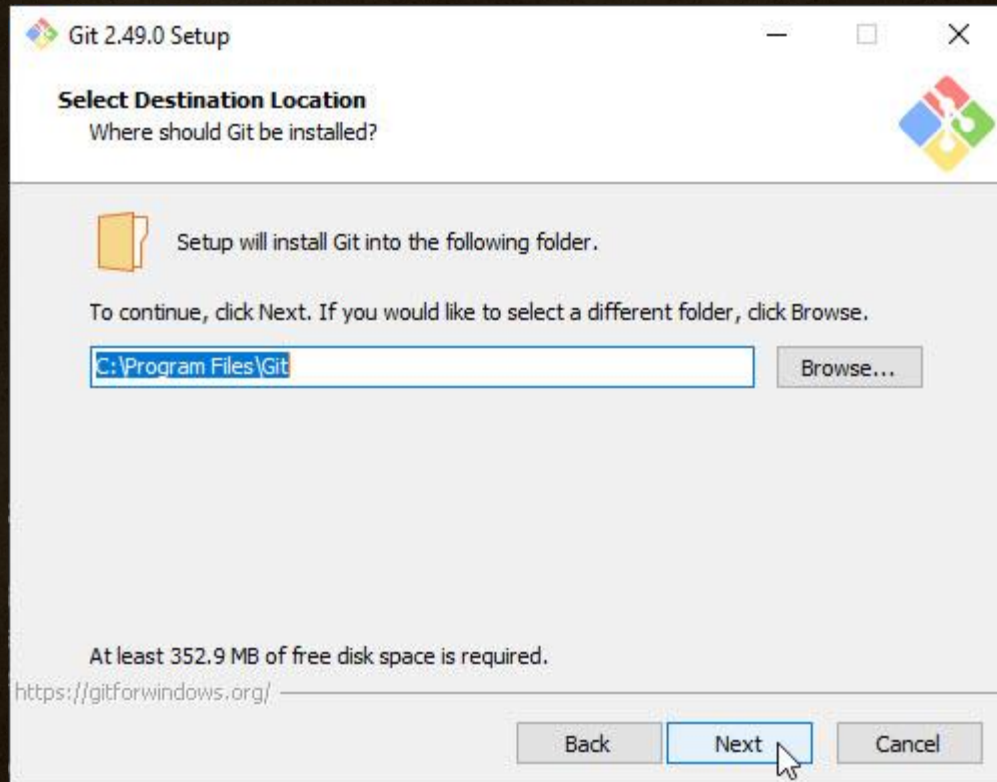
<https://git-scm.com/downloads/win>

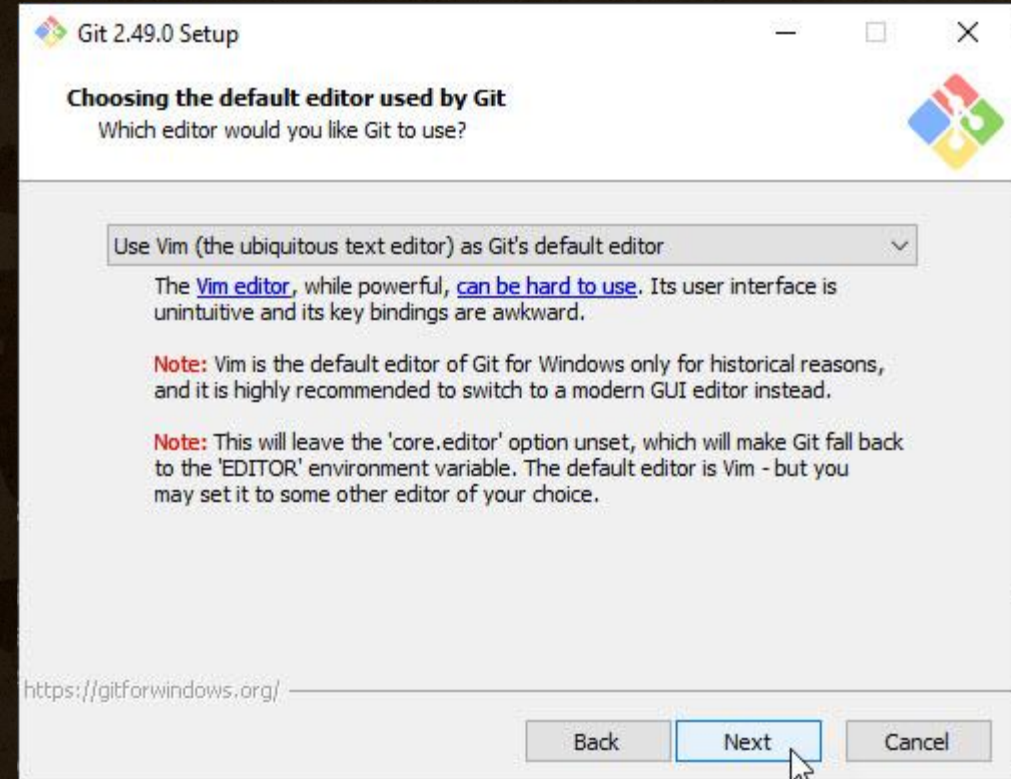
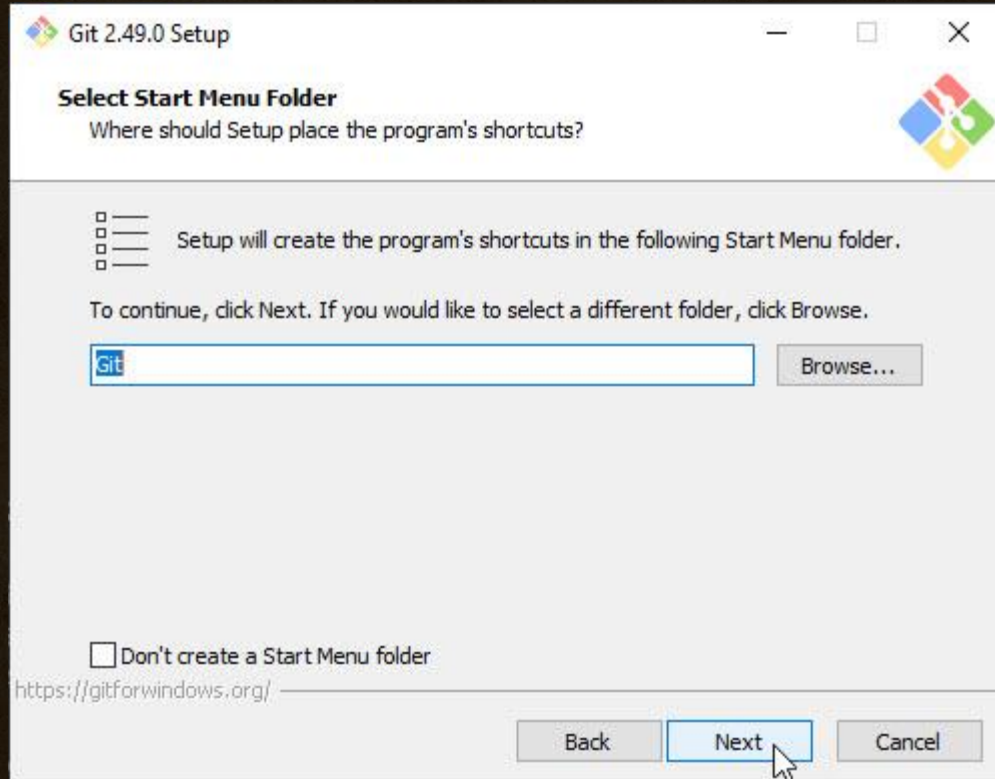


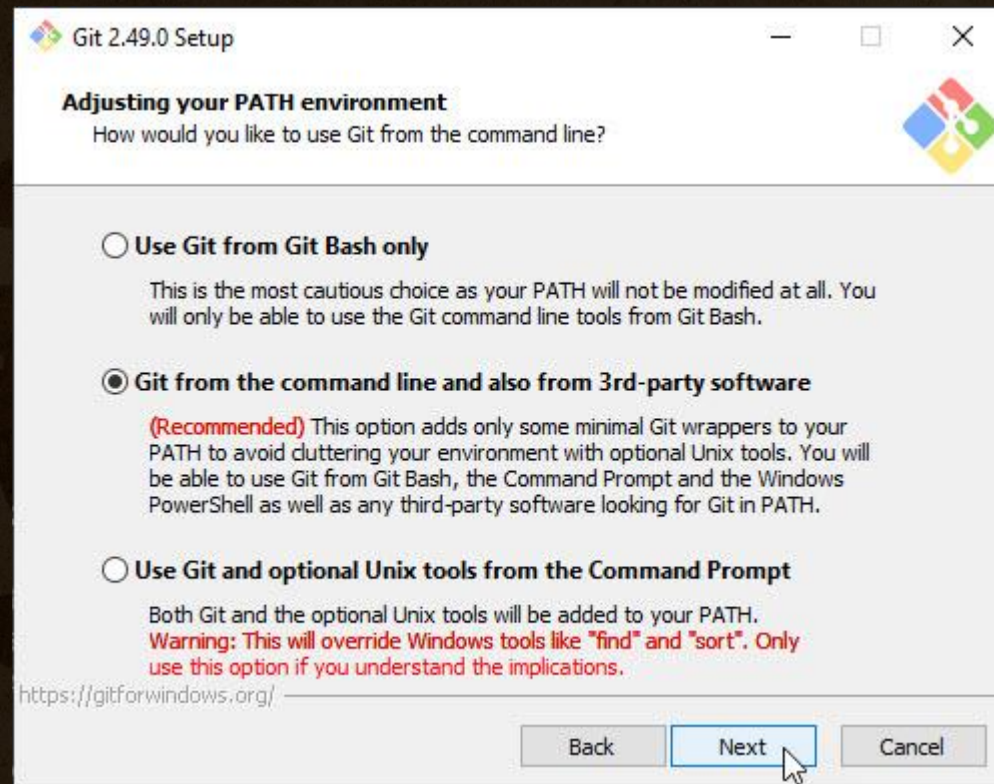
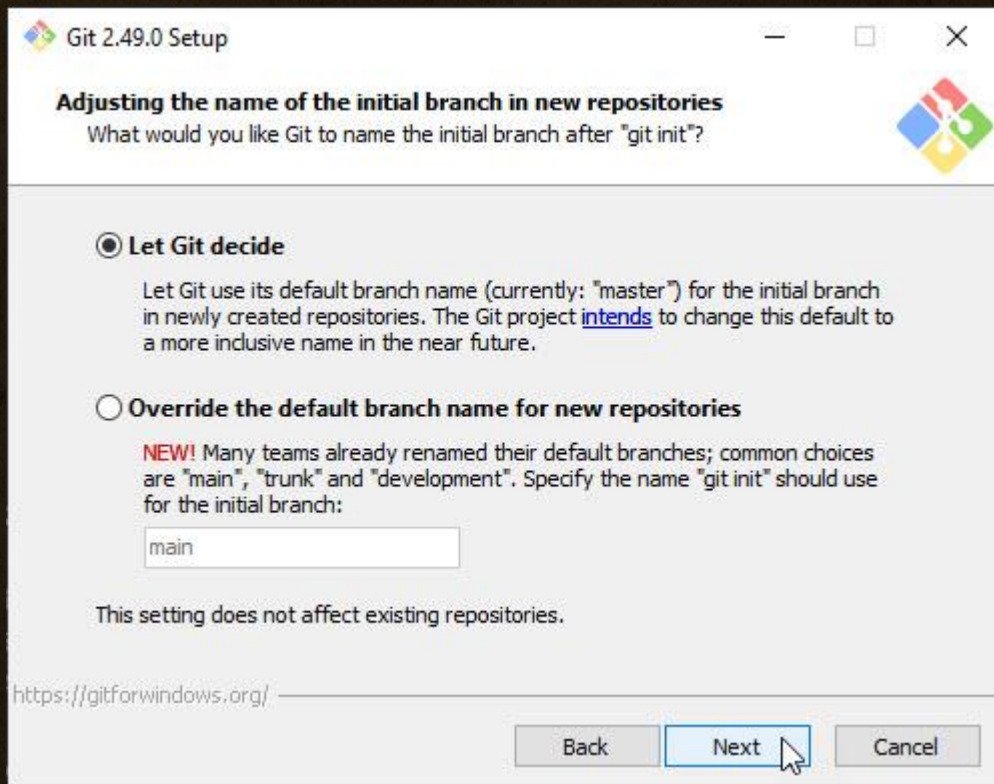
สถาบัน

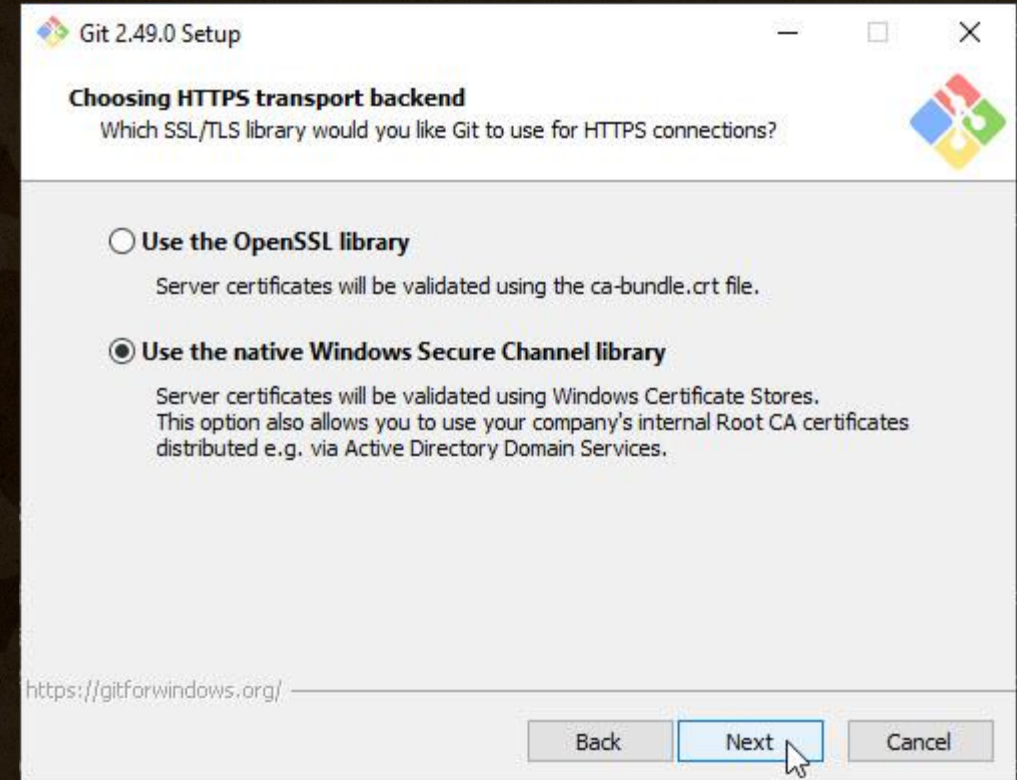
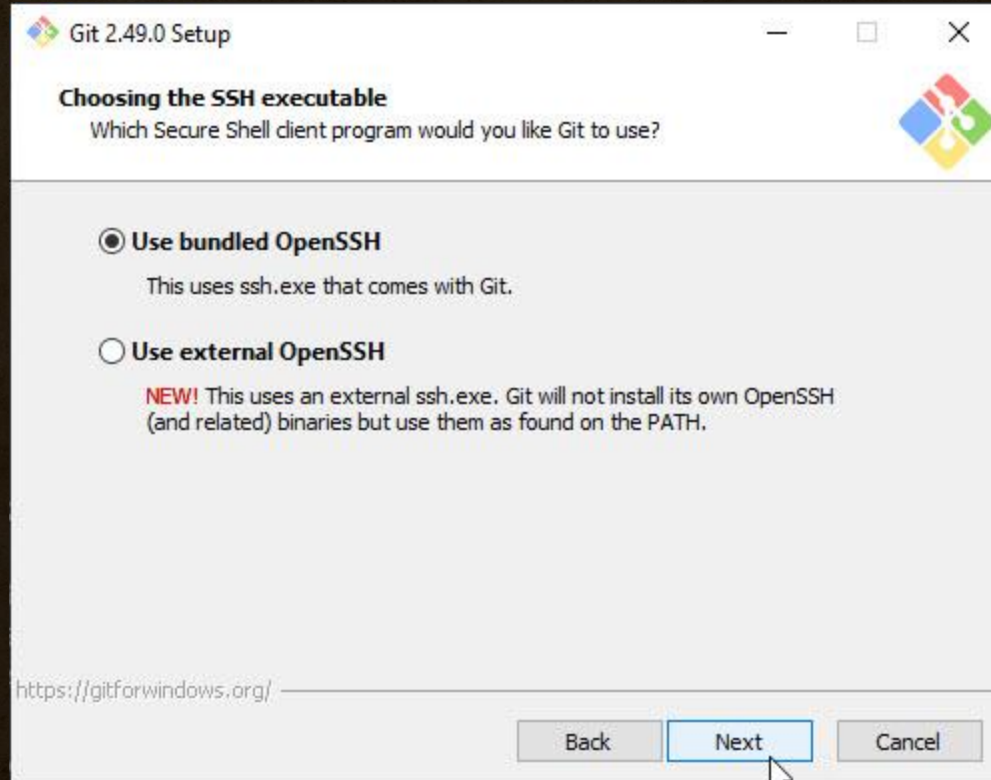
nius.co.th

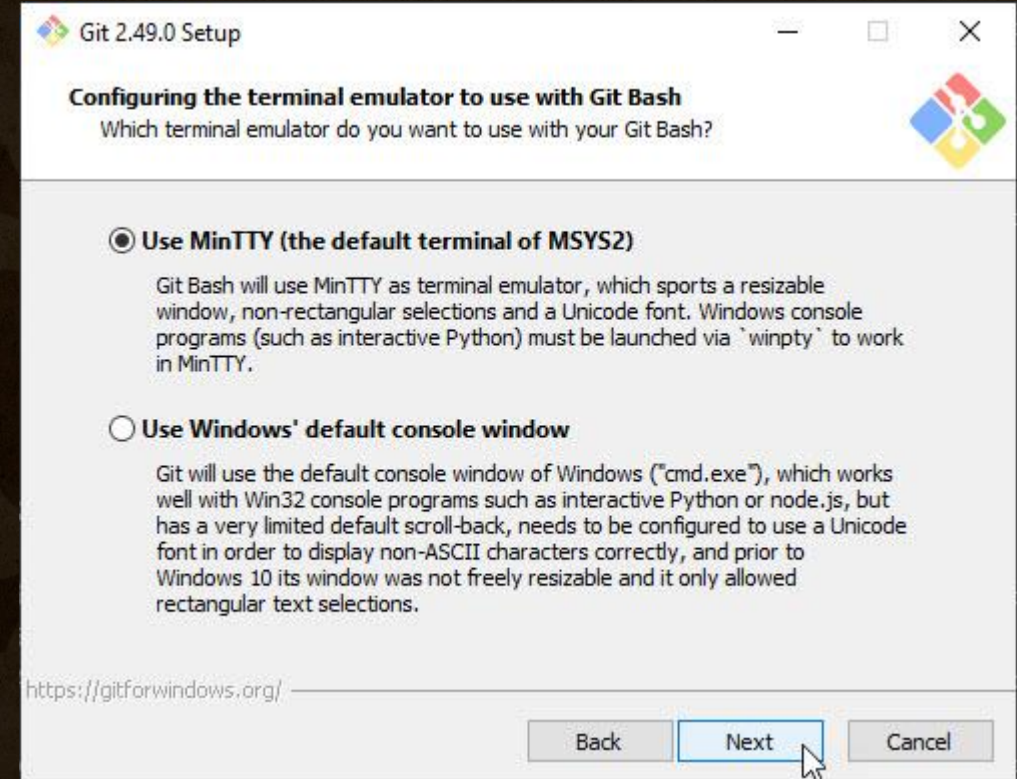
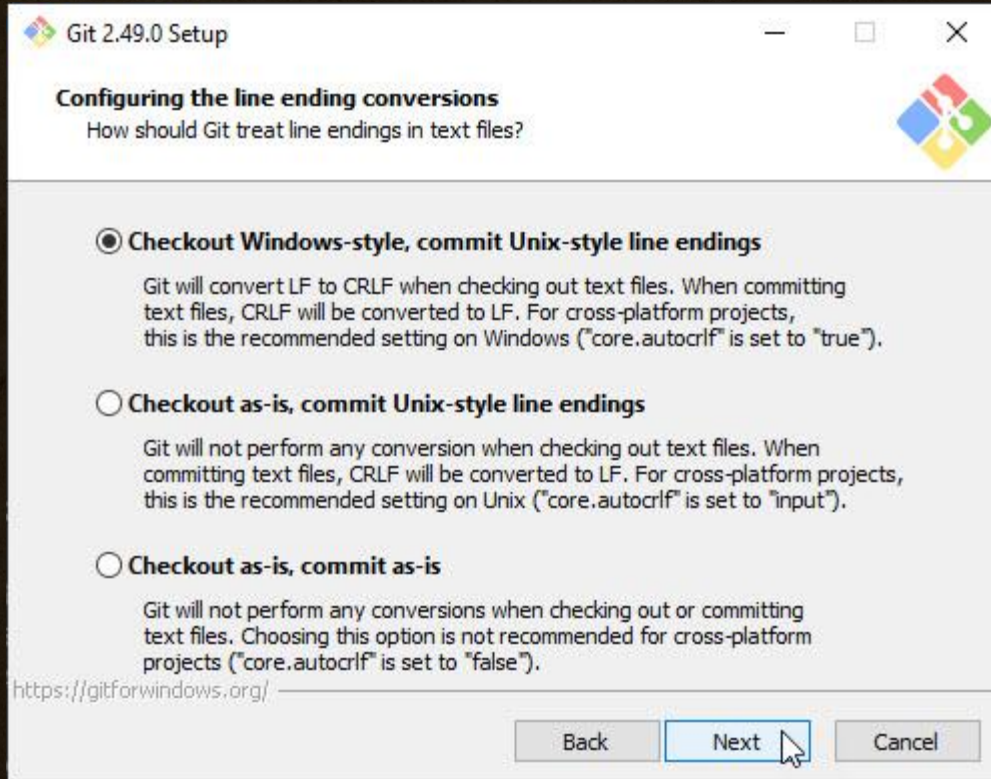


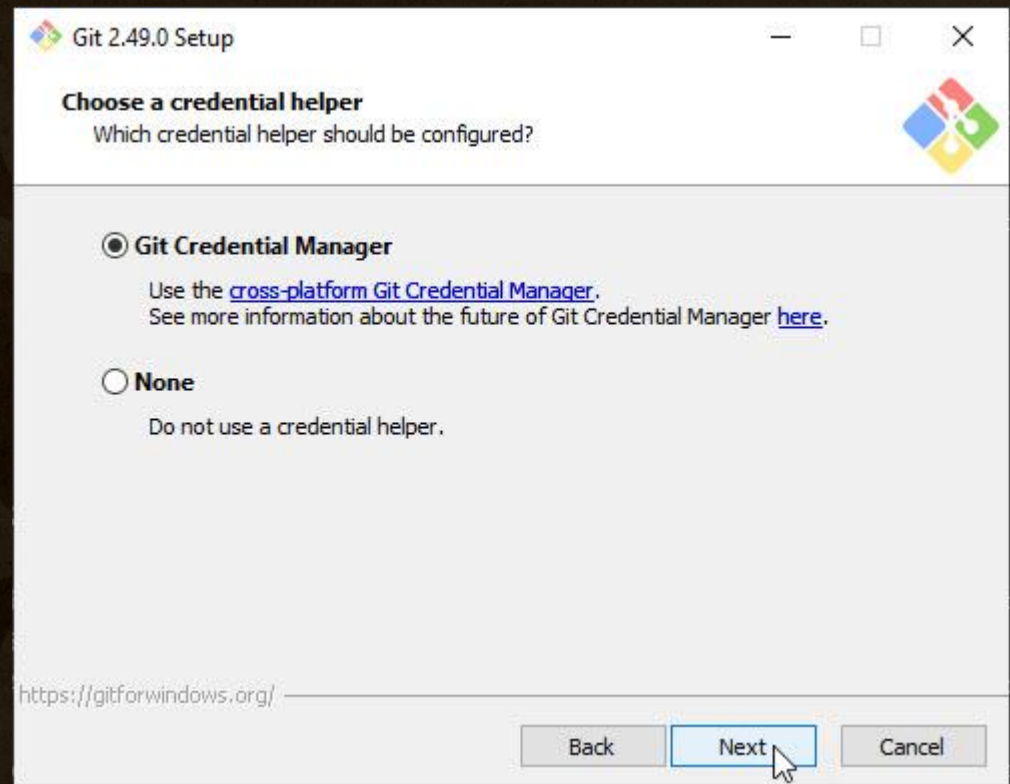
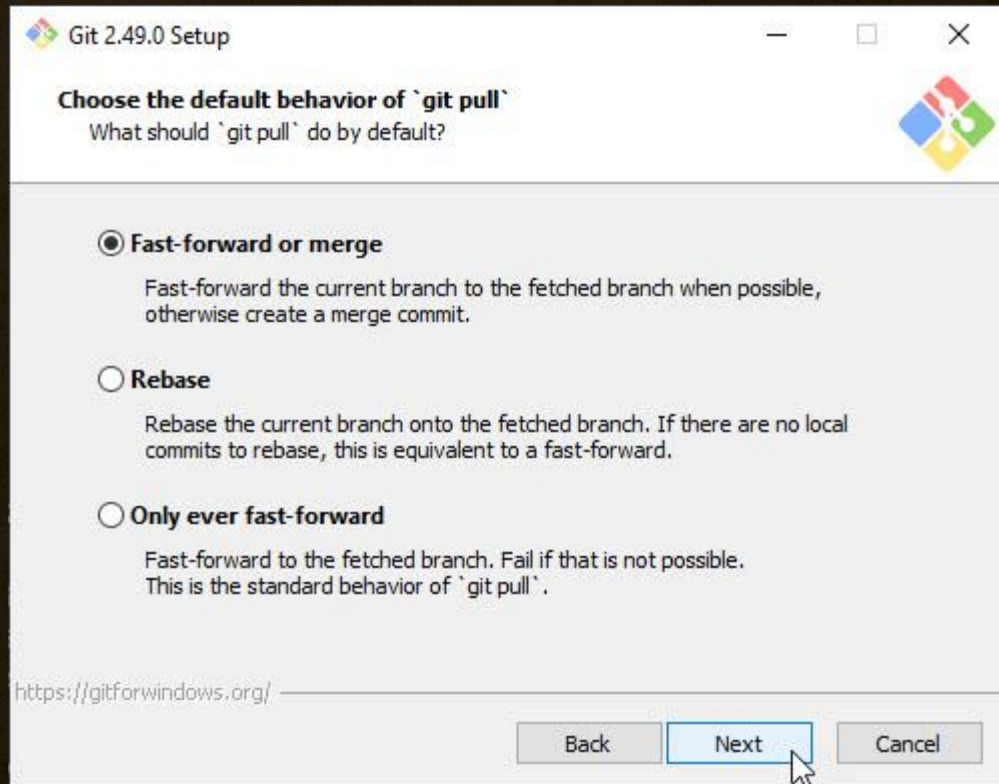


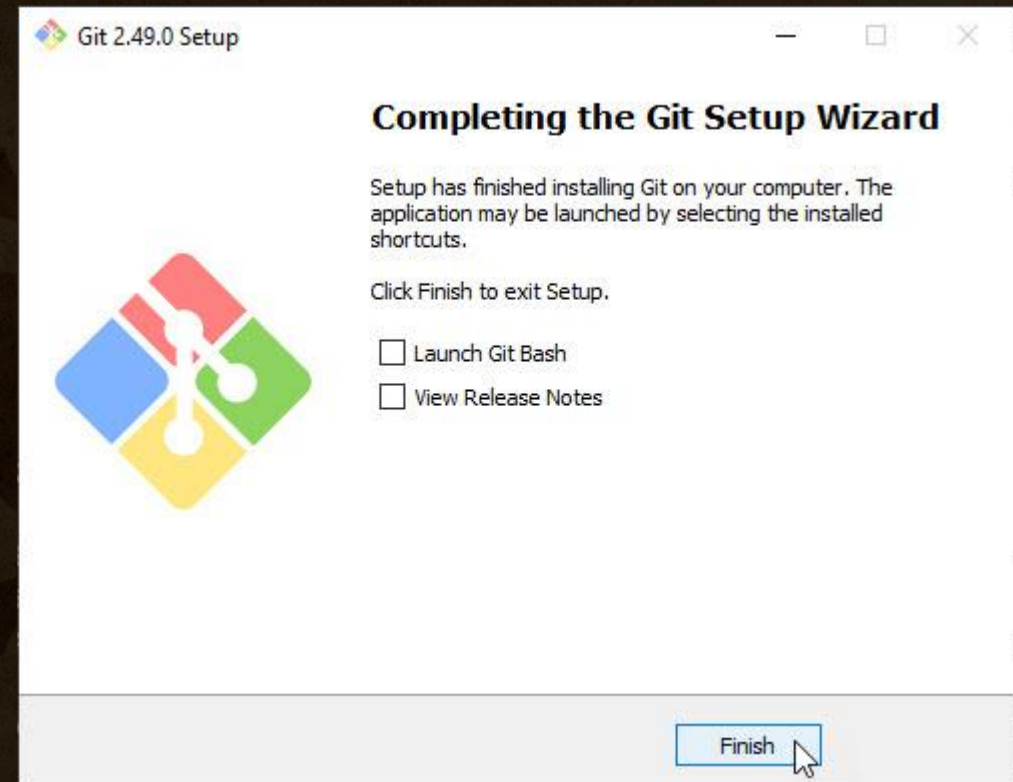
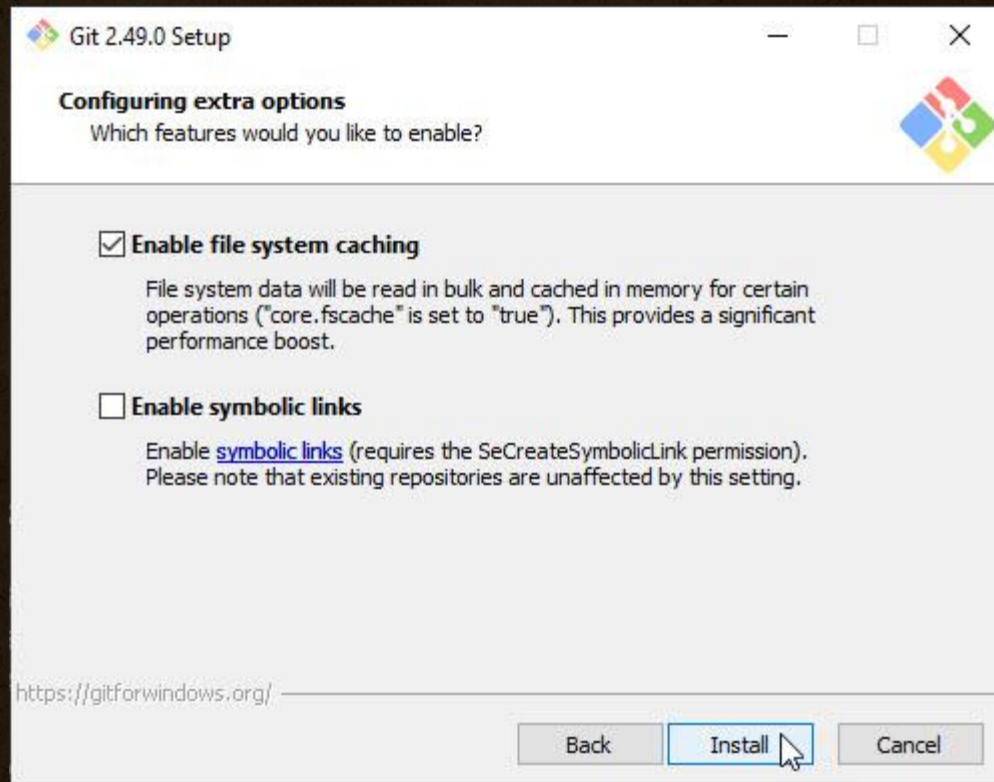








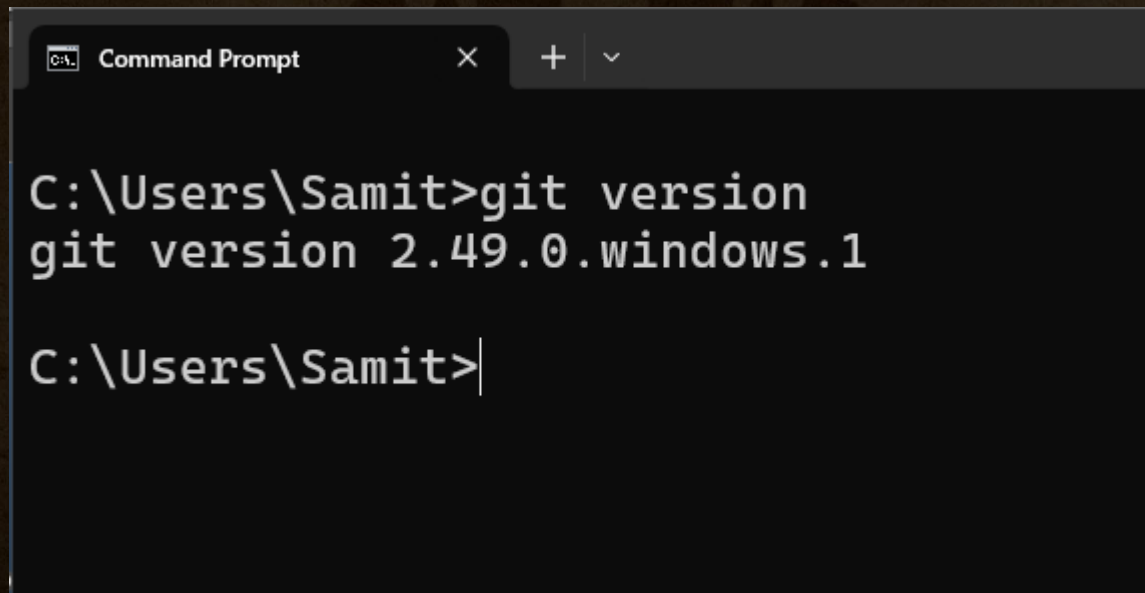




หลังติดตั้งสำเร็จทดสอบด้วยคำสั่ง

git --version

หากพบเวอร์ชันดังกล่าว ถือว่าติดตั้งเรียบร้อยแล้วพร้อมใช้งาน



```
Command Prompt
C:\Users\Samit>git version
git version 2.49.0.windows.1
C:\Users\Samit>
```



การตรวจสอบความเรียบร้อยของเครื่องมือที่ติดตั้งบน Windows / Mac OS / Linux

เปิด Command Prompt บน Windows หรือ Terminal บน Mac ขึ้นมาป้อนคำสั่งดังนี้

Rustup for Windows / MacOS / Linux

```
rustc --version
```

```
cargo --version
```

Visual Studio Code

```
code --version
```

Git

```
git version
```





การใช้งาน Cargo Package Manager



สถาบันไอทีจีเนียส

www.itgenius.co.th



Cargo คืออะไร?

Cargo คือเครื่องมือจัดการแพ็คเกจและ build system ที่มาพร้อมกับภาษา Rust โดยอัตโนมัติ ถ้าใช้ Rust → แทบจะ “ขาด Cargo ไม่ได้”



ความสามารถหลักของ Cargo

ความสามารถ

รายละเอียด



สร้างโปรเจกต์ใหม่

`cargo new`, `cargo init`



คอมไพล์โปรเจกต์

`cargo build`, `cargo run`



รัน unit tests

`cargo test`



จัดการ dependencies

เพิ่ม/ลบ crates ผ่าน `Cargo.toml`



จัดการรูปแบบโค้ด

`cargo fmt`



วิเคราะห์ style โค้ด

`cargo clippy`



สร้างเอกสาร API

`cargo doc`





cargo new และ cargo init

cargo new – สร้างโปรเจกต์ใหม่พร้อมโฟลเดอร์

ใช้เมื่อ:

ต้องการสร้าง โปรเจกต์ Rust ใหม่ พร้อมสร้างโฟลเดอร์และโครงสร้างไฟล์ทั้งหมดอัตโนมัติ

ตัวอย่าง:

```
bash
cargo new hello_project
```

ผลลัพธ์:

```
css
hello_project/
├── Cargo.toml
└── src/
    └── main.rs
```

หมายเหตุ:

- จะสร้างโฟลเดอร์ชื่อ `hello_project`
- ใส่ไฟล์ `Cargo.toml` ให้พร้อม
- มีโฟลเดอร์ `src` และ `main.rs` พร้อมโค้ด Hello World

cargo init – เริ่มต้นใช้งานในโฟลเดอร์ที่มีอยู่แล้ว

ใช้เมื่อ:

ต้องการ เริ่มใช้ Rust ในโฟลเดอร์ปัจจุบัน (ที่อาจมีไฟล์หรือเป็นโฟลเดอร์เปล่าอยู่แล้ว)

ตัวอย่าง:

```
bash
mkdir my_folder
cd my_folder
cargo init
```

ผลลัพธ์ใน `my_folder/`:

```
css
Cargo.toml
src/
└── main.rs
```

ข้อควรระวัง:

- ไม่สร้างโฟลเดอร์ใหม่
- ใช้กับโฟลเดอร์ที่มีอยู่แล้วเท่านั้น
- เหมาะกับการแปลงโปรเจกต์ที่มีอยู่ หรือเริ่มโปรเจกต์จากใดเรกทอรีปัจจุบัน



การสร้างโปรเจกต์ “Hello World” ใน Rust

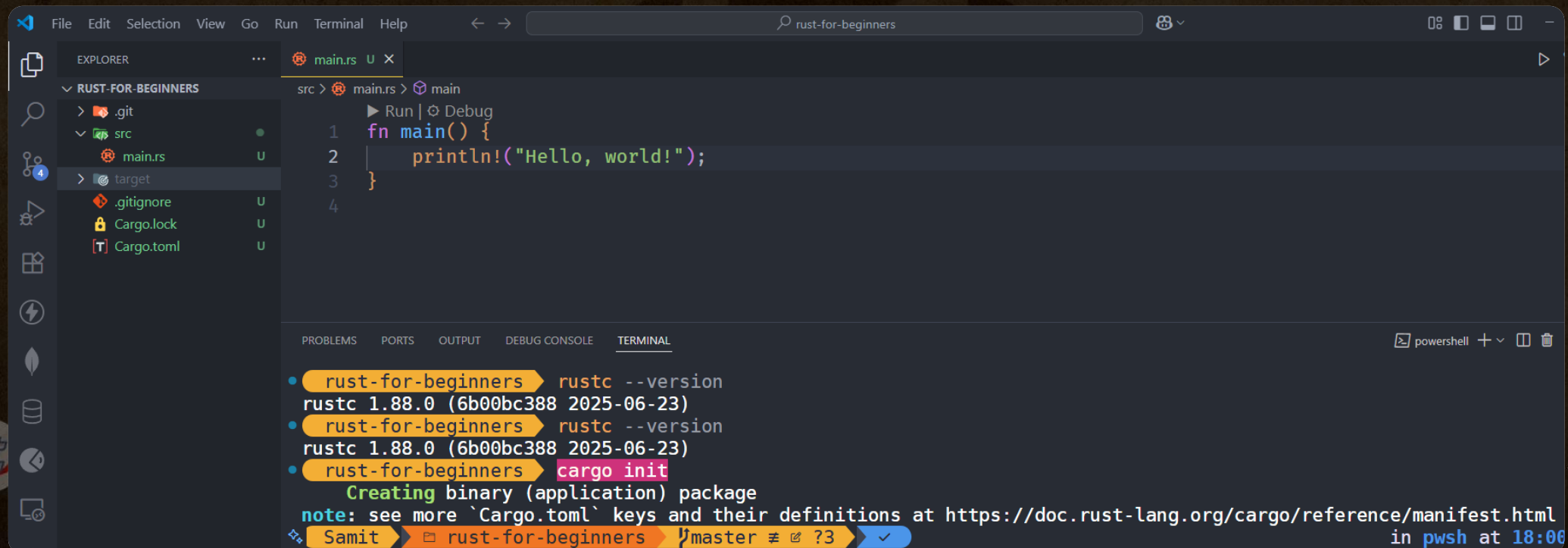


Step 1: สร้างโฟลเดอร์สำหรับจัดเก็บโปรเจกต์

File Explorer path: This PC > Local Disk (C:) > TrainingWorkshop > **rust-for-beginners**

Step 2: เปิด folder เข้า vscode และทำการ init cargo เข้าไปในโปรเจกต์

cargo init



```
File Edit Selection View Go Run Terminal Help
rust-for-beginners

EXPLORER
RUST-FOR-BEGINNERS
  .git
  src
    main.rs
  target
  .gitignore
  Cargo.lock
  Cargo.toml

main.rs
1 fn main() {
2     println!("Hello, world!");
3 }
4

TERMINAL
rust-for-beginners rustc --version
rustc 1.88.0 (6b00bc388 2025-06-23)
rust-for-beginners rustc --version
rustc 1.88.0 (6b00bc388 2025-06-23)
rust-for-beginners cargo init
Creating binary (application) package
note: see more `Cargo.toml` keys and their definitions at https://doc.rust-lang.org/cargo/reference/manifest.html
Samit rust-for-beginners master
```

Step 3: ทดสอบ Run โปรแกรม ด้วยคำสั่ง cargo run

cargo run

The screenshot shows an IDE with the following components:

- EXPLORER:** Displays the project structure for 'RUST-FOR-BEGINNERS'. The 'src' folder is expanded, showing 'main.rs'. The 'target' folder is also expanded, showing 'debug' and 'build' subfolders.
- main.rs:** Contains the following code:

```
1 fn main() {  
2     println!("Hello, world!");  
3 }  
4
```
- TERMINAL:** Shows the output of the 'cargo run' command:

```
rust-for-beginners cargo run  
Compiling rust-for-beginners v0.1.0 (C:\TrainingWorkshop\rust-for-beginners)  
Finished `dev` profile [unoptimized + debuginfo] target(s) in 0.64s  
Running `target\debug\rust-for-beginners.exe`  
Hello, world!
```



อธิบายความหมายของไฟล์ main.rs

```
main.rs U X
src > main.rs > main
  ▶ Run | ⚙ Debug
1  fn main() {
2      println!("Hello, world!");
3  }
4
```

🔍 คำอธิบายที่ละบรรทัด

บรรทัด

คำอธิบาย

`fn main()`

บอกให้ Rust รู้ว่า นี่คือจุดเริ่มต้นของโปรแกรม

คำว่า `fn` = *function*

`main` คือชื่อของฟังก์ชันหลัก คล้าย `int main()` ใน C/C++

`{ ... }`

วงเล็บปีกกา เปิด-ปิดขอบเขตของบล็อกโค้ด ที่อยู่ในฟังก์ชัน `main`

`println!("Hello, world!");`

คำสั่ง พิมพ์ข้อความออกทางหน้าจอ

- `println!` คือ macro (มี `!` ต่อท้าย)
- `"Hello, world!"` คือ string literal
- เครื่องหมาย `;` คือการปิดท้าย statement



อธิบายความหมายของไฟล์ main.rs

```
main.rs U X
src > main.rs > main
  ▶ Run | ⚙ Debug
1  fn main() {
2      println!("Hello, world!");
3  }
4
```

🔑 รายละเอียดเพิ่มเติม:

◆ fn – Function Declaration

- คำสั่งที่ใช้สร้าง ฟังก์ชัน ใน Rust
- ไม่มี `public`, `private` แบบภาษาอื่นในฟังก์ชัน `main`

◆ main – Entry Point

- เป็นฟังก์ชันพิเศษในทุกโปรแกรม Rust
- ระบบจะเริ่มรันจากตรงนี้เมื่อสั่ง `cargo run`

◆ println! – Macro ไม่ใช่ Function

- ทำหน้าที่พิมพ์ข้อความ (คล้าย `printf`, `console.log`)
- ใช้ `!` เพราะเป็น macro (คล้าย template ที่ compile-time)
- รองรับการ format เช่น:

rust

📄 Copy ✎ Edit

```
println!("My name is {} and I am {} years old.", "Samit", 40)
```



สถาบันไอทีจีเนียส

อธิบายความหมายของไฟล์ main.rs

? ต้องตั้งชื่อไฟล์ว่า `main.rs` เสมอหรือไม่?

✓ คำตอบ:

ไม่จำเป็นเสมอไป แต่ ขึ้นอยู่กับบริบทของการทำงาน ดังนี้:

บริบทของโปรเจกต์	ชื่อไฟล์หลักที่ต้องใช้
Binary crate (แอปพลิเคชันทั่วไป)	<code>src/main.rs</code>
Library crate (สร้างเป็น lib)	<code>src/lib.rs</code>
Multiple binaries	<code>src/bin/<ชื่อไฟล์>.rs</code> เช่น <code>src/bin/server.rs</code> , <code>src/bin/client.rs</code>

📌 สรุป: ถ้าเป็นแอปทั่วไปใช้ `cargo new` แบบ binary → Rust จะหา `main.rs` เป็นจุด entry point แต่ถ้าใช้ `lib.rs` → จะไม่มี `fn main()` แต่มี `pub fn` ต่าง ๆ แทน



อธิบายความหมายของไฟล์ main.rs

? แล้ว `fn main()` จำเป็นต้องอยู่ใน `main.rs` หรือไม่?

✓ คำตอบ:

- ในกรณีของ `binary crate` → `fn main()` ต้องอยู่ใน `src/main.rs` หรือ `src/bin/*.rs`
- ถ้ามีหลายไฟล์ที่มี `fn main()` ต้องใส่ไว้ใน `src/bin/` และเรียก run แบบกำหนดชื่อ เช่น:

bash

Copy Edit

```
cargo run --bin client  
cargo run --bin server
```



อธิบายความหมายของไฟล์ main.rs

? ถ้ามีหลาย `fn main()` ในไฟล์เดียว จะเกิดอะไรขึ้น?

✗ ไม่สามารถมีหลาย `fn main()` ในไฟล์เดียวกันได้

Rust จะ error แบบนี้ตอน compile:

```
perl
```

Copy Edit

```
error[E0428]: the name `main` is defined multiple times
```

🚩 เพราะชื่อ `main` ถูกใช้เป็นจุด entry point ได้เพียงครั้งเดียวในหนึ่ง binary



อธิบายความหมายของไฟล์ main.rs

✓ ตัวอย่างการใช้งานหลาย main:

โครงสร้าง:

CSS

Copy Edit

src/

```
|— main.rs      (fn main() ← default run)
|— bin/
|   |— server.rs (fn main() ← run ด้วย `cargo run --bin server`)
|   |— client.rs (fn main() ← run ด้วย `cargo run --bin client`)
```

คำสั่ง:

bash

Copy Edit

```
cargo run          # จะรัน main.rs
cargo run --bin server # จะรัน server.rs
```



อธิบายความหมายของไฟล์ main.rs



สรุปโดยรวม

คำถาม

ต้องชื่อ `main.rs` ไหม

ต้องมี `fn main()` ไหม

มีหลาย `fn main()` ได้ไหม

สั่งรันแต่ละไฟล์อย่างไร

คำตอบ

✓ ถ้าเป็น binary หลักของโปรเจกต์

✓ ถ้าเป็นโปรแกรมที่รันได้ (ไม่ใช่ library)

✓ ถ้าแยกไฟล์ใน `src/bin/`

✗ ถ้าอยู่ในไฟล์เดียวกัน

ใช้ `cargo run --bin <ชื่อไฟล์>`



ตัวอย่างการมี fn main อยู่ทีอื่นในโปรเจกต์

โครงสร้างโปรเจกต์

text

 Copy  Edit

```
myapp/  
├─ Cargo.toml  
└─ src/  
    ├─ main.rs          # (optional)  
    └─ bin/  
        ├─ server.rs    # binary 1  
        └─ client.rs    # binary 2
```



ตัวอย่างการมี fn main อยู่ทีอื่นในโปรเจกต์



src/bin/server.rs

rust

Copy Edit

```
fn main() {  
    println!("🚀 Server is starting...");  
  
    // สมมติ: ทำงาน server เช่น listen port  
    for i in 1..=3 {  
        println!("🟢 Listening on port 808{}", i)  
    }  
  
    println!("✅ Server ready.")  
}
```



ตัวอย่างการมี fn main อยู่ทีอื่นในโปรเจกต์



src/bin/client.rs

rust

Copy Edit

```
fn main() {  
    println!("🔔 Client is connecting...");  
  
    // สมมติ: จำลองการ connect ไปยัง server  
    let server_address = "127.0.0.1:8080"  
    println!("🖱️ Connected to server at {}", server_address)  
  
    println!("📡 Sending request...");  
    println!("📡 Received response: 200 OK")  
}
```



สถาบันไอทีจีเนียส

www.itgenius.co.th

คำสั่งใช้ในการ run แต่ละ binary

คำสั่งสำหรับรันแต่ละ binary

bash

 Copy  Edit

```
cargo run --bin server  
cargo run --bin client
```

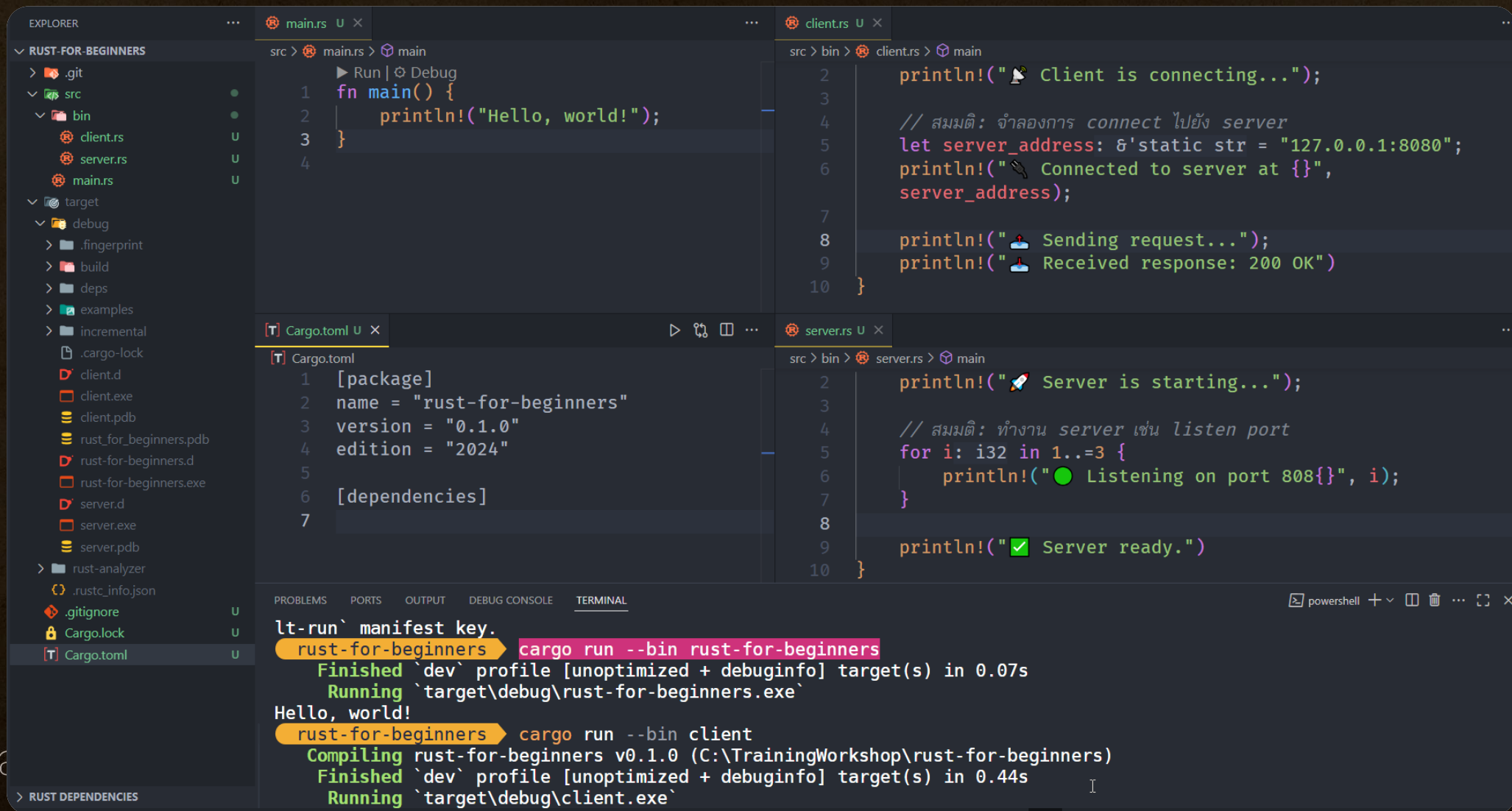
หมายเหตุ: ไม่มีความจำเป็นต้องมี `src/main.rs` ก็ได้ ถ้าไม่ได้ใช้ `cargo run` เลย ๆ

ข้อดีของการใช้ `src/bin/*.rs`

- สามารถแยก logic แต่ละ binary ชัดเจน
- ใช้ร่วมกับ library ภายในโปรเจกต์ได้ เช่น แยก logic common ไว้ใน `src/lib.rs`
- ดีสำหรับงานแบบ CLI tools, server/client tools, microservices



คำสั่งใช้ในการ run แต่ละ binary



The screenshot displays an IDE with the following components:

- EXPLORER:** Shows the project structure for 'RUST-FOR-BEGINNERS', including files like `client.rs`, `server.rs`, and `main.rs`.
- main.rs:** Contains a `main` function that prints "Hello, world!".

```
1 fn main() {  
2     println!("Hello, world!");  
3 }  
4
```
- client.rs:** Contains code for a client connecting to a server.

```
2 println!("Client is connecting...");  
3  
4 // สมมติ: จำลองการ connect ไปยัง server  
5 let server_address: &'static str = "127.0.0.1:8080";  
6 println!("Connected to server at {}",  
7     server_address);  
8  
9 println!("Sending request...");  
10 println!("Received response: 200 OK")  
11 }
```
- server.rs:** Contains code for a server listening on port 8080.

```
2 println!("Server is starting...");  
3  
4 // สมมติ: ทำงาน server เช่น listen port  
5 for i: i32 in 1..=3 {  
6     println!("Listening on port 808{}", i);  
7 }  
8  
9 println!("Server ready.")  
10 }
```
- Cargo.toml:** Defines the package name as "rust-for-beginners" and edition as "2024".

```
1 [package]  
2 name = "rust-for-beginners"  
3 version = "0.1.0"  
4 edition = "2024"  
5  
6 [dependencies]  
7
```
- TERMINAL:** Shows the execution of the binaries.

```
lt-run` manifest key.  
rust-for-beginners cargo run --bin rust-for-beginners  
Finished `dev` profile [unoptimized + debuginfo] target(s) in 0.07s  
Running `target\debug\rust-for-beginners.exe`  
Hello, world!  
rust-for-beginners cargo run --bin client  
Compiling rust-for-beginners v0.1.0 (C:\TrainingWorkshop\rust-for-beginners)  
Finished `dev` profile [unoptimized + debuginfo] target(s) in 0.44s  
Running `target\debug\client.exe`
```


คำสั่งใช้ในการ run แต่ละ binary

 สรุป:

ชนิดไฟล์	binary name	วิธีรัน
src/main.rs	rust-for-beginners	cargo run --bin rust-for-beginners
src/bin/client.rs	client	cargo run --bin client
src/bin/server.rs	server	cargo run --bin server



คำสั่งใช้ในการ run แต่ละ binary

หากต้องการให้ระบบ default ไปที่ binary ใด binary หนึ่ง สามารถตั้งใน `Cargo.toml` ได้ด้วย `default-run` เช่น:

toml

Copy Edit

```
[package]
name = "rust-for-beginners"
version = "0.1.0"
edition = "2021"
default-run = "server"
```

จากนั้นใช้แค่:

bash

Copy Edit

```
cargo run
```

ก็จะรัน `server.rs` ได้เลยครับ 



อธิบายความหมายของไฟล์ Cargo.toml



อธิบายความหมายของไฟล์ Cargo.toml

◆ โครงสร้างตัวอย่าง:

toml

[package]

name = "rust-for-beginners"

version = "0.1.0"

edition = "2024"

[dependencies]

ส่วนที่ 1: [package]

เป็น section สำหรับกำหนดข้อมูล เกี่ยวกับตัวโปรเจกต์

คีย์

ความหมาย

name

ชื่อของโปรเจกต์ หรือ crate ชื่อนี้จะใช้ตอน publish crate ไปยัง crates.io

version

เวอร์ชันของโปรเจกต์ในรูปแบบ [Semantic Versioning](https://semver.org/) เช่น 0.1.0 , 1.0.0

edition

เวอร์ชันของ Rust edition ที่ใช้ เช่น 2018 , 2021 , หรือ 2024 (แนะนำใช้ของปีล่าสุด)

◆หมายเหตุเพิ่มเติม:

- Rust edition จะกำหนด syntax และ behavior บางอย่าง (เช่น `async` ถูกเพิ่มใน edition 2018)
- ใช้ 2024 เฉพาะใน nightly หรือเวอร์ชันล่าสุดเท่านั้น (ณ ตอนตอนนี้ ยังไม่ stable)



อธิบายความหมายของไฟล์ Cargo.toml

◆ โครงสร้างตัวอย่าง:

toml

[package]

name = "rust-for-beginners"

version = "0.1.0"

edition = "2024"

[dependencies]

✖ ส่วนที่ 2: [dependencies]

เป็น section สำหรับกำหนด crate ภายนอกที่โปรเจกต์นี้จะใช้งาน

เช่น:

toml

Copy Edit

[dependencies]

serde = "1.0"

tokio = { version = "1", features = ["full"] }

ตัวอย่าง

ความหมาย

serde = "1.0"

ดึง crate `serde` เวอร์ชัน 1.0 จาก crates.io

tokio = { version = "1", features =
["full"] }

กำหนดเวอร์ชันและเปิด feature เสริมของ crate

หากไม่มีอะไรใน [dependencies] หมายความว่า ยังไม่มีการใช้งาน crate ภายนอก ในโปรเจกต์นี้



สถาบันไอทีจีเนียส

www.itgenius.co.th

อธิบายความหมายของไฟล์ Cargo.toml

◆ โครงสร้างตัวอย่าง:

toml

[package]

name = "rust-for-beginners"

version = "0.1.0"

edition = "2024"

[dependencies]



ไฟล์ Cargo.toml ที่สมบูรณ์ อาจมี section เพิ่มเติม เช่น:

- [dev-dependencies] สำหรับ crate ที่ใช้เฉพาะตอนเขียน test หรือ dev เท่านั้น
- [features] สำหรับกำหนด feature flags
- [profile] สำหรับตั้งค่า build profile เช่น debug หรือ release
- [workspace] ถ้าเป็น multi-crate workspace



สรุปภาพรวม

Section

หน้าที่หลัก

[package]

ข้อมูล metadata ของโปรเจกต์

[dependencies]

crate ภายนอกที่ต้องใช้

(อื่น ๆ)

profiles, features, dev-dependencies (ในโปรเจกต์ใหญ่)



สถาบันไอทีจีเนียส

www.itgenius.co.th

อธิบายเพิ่มเติม feature ใน dependency

ความหมายของแต่ละส่วน

toml

 Copy  Edit

```
tokio = { version = "1", features = ["full"] }
```

ส่วน

ความหมาย

tokio

คือชื่อ crate ที่ต้องการใช้งาน (runtime async ยอดนิยมใน Rust)

version = "1"

ต้องการใช้เวอร์ชัน 1.x.x ล่าสุดของ tokio (ไม่ระบุ patch)

features = ["full"]

เปิดใช้ feature set ที่ชื่อว่า `full` ซึ่งเป็นชุดรวมของฟีเจอร์ย่อยทั้งหมดใน tokio

รายละเอียดของ `features = ["full"]`

Tokio เป็น crate ที่ออกแบบให้ modular มาก เพื่อให้เบาและเร็ว โดย default จะเปิดแค่ core async runtime เท่านั้น

เมื่อเพิ่ม `features = ["full"]` จะเปิดทั้งหมด เช่น:

- `rt-multi-thread` : runtime แบบ multi-threaded
- `macros` : ใช้ `#[tokio::main]`, `#[tokio::test]`
- `time` : ใช้ delay, timeout, sleep ฯลฯ
- `net` : TCP/UDP networking
- `io-util` : read/write stream
- `signal` : ดักจับสัญญาณ OS
- `sync` : channel แบบ async (เช่น `mpsc`)
- และอื่น ๆ

ตัวอย่างการใช้งานหลังใส่ full:

rust

 Copy  Edit

```
#[tokio::main]
async fn main() {
    println!("Hello from Tokio async runtime!")
}
```

หากไม่เปิด `features = ["full"]` เราจะใช้ macro `#[tokio::main]` ไม่ได้ และ runtime บางแบบจะไม่สามารถใช้งานได้



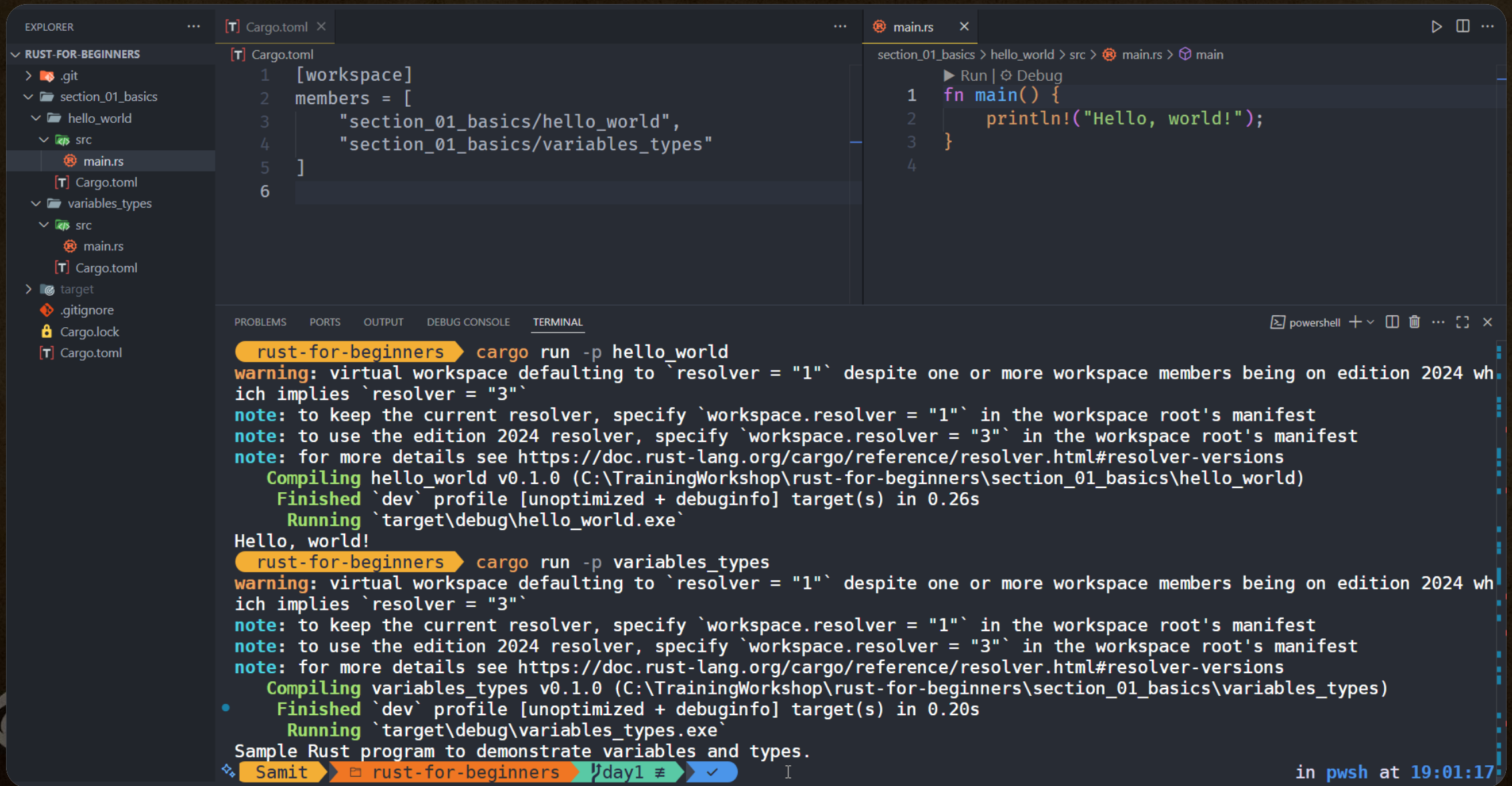


ไวยากรณ์พื้นฐานและ data types



cargo run -p hello_world

cargo run -p variables_types



```
[T] Cargo.toml
1 [workspace]
2 members = [
3     "section_01_basics/hello_world",
4     "section_01_basics/variables_types"
5 ]
6
```

```
section_01_basics > hello_world > src > main.rs > main
1 fn main() {
2     println!("Hello, world!");
3 }
4
```

```
rust-for-beginners > cargo run -p hello_world
warning: virtual workspace defaulting to `resolver = "1"` despite one or more workspace members being on edition 2024 which implies `resolver = "3"`
note: to keep the current resolver, specify `workspace.resolver = "1"` in the workspace root's manifest
note: to use the edition 2024 resolver, specify `workspace.resolver = "3"` in the workspace root's manifest
note: for more details see https://doc.rust-lang.org/cargo/reference/resolver.html#resolver-versions
   Compiling hello_world v0.1.0 (C:\TrainingWorkshop\rust-for-beginners\section_01_basics\hello_world)
   Finished `dev` profile [unoptimized + debuginfo] target(s) in 0.26s
   Running `target\debug\hello_world.exe`
Hello, world!

rust-for-beginners > cargo run -p variables_types
warning: virtual workspace defaulting to `resolver = "1"` despite one or more workspace members being on edition 2024 which implies `resolver = "3"`
note: to keep the current resolver, specify `workspace.resolver = "1"` in the workspace root's manifest
note: to use the edition 2024 resolver, specify `workspace.resolver = "3"` in the workspace root's manifest
note: for more details see https://doc.rust-lang.org/cargo/reference/resolver.html#resolver-versions
   Compiling variables_types v0.1.0 (C:\TrainingWorkshop\rust-for-beginners\section_01_basics\variables_types)
   Finished `dev` profile [unoptimized + debuginfo] target(s) in 0.20s
   Running `target\debug\variables_types.exe`
Sample Rust program to demonstrate variables and types.
```

Samit rust-for-beginners day1 in pwsh at 19:01:17